© 2026 ANSYS, Inc. or affiliated companies
Unauthorized use, distribution, or duplication prohibited.

Ansys Sphinx Theme

ANSYS, Inc.
Southpointe
2600 Ansys Drive
Canonsburg, PA 15317
ansysinfo@ansys.com
http://www.ansys.com
(T) 724-746-3304
(F) 724-514-9494

Nov 05, 2026

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001:2015 companies.

CONTENTS

The Ansys Sphinx Theme is a custom Ansys-branded theme for use with Sphinx, a documentation generator for creating project documentation from reStructuredText source files.

This theme is specifically tailored for documentation related to Ansys projects helping to ensure consistency in its look and feel. Various useful extensions are included in the theme to make documentation more appealing and user-friendly.

Getting started Learn how to install the Ansys Sphinx Theme.

Getting started User guide Learn how to use the capabilities and features of this theme.

User guide Examples Explore examples that show how to integrate third-party extensions with this theme.

Examples Contribute Learn how to contribute to the Ansys Sphinx Theme.

Contribute Changelog View the changelog for this project.

Release notes

CHAPTER

ONE

GETTING STARTED

How to install Learn how to download and install the theme.

Installation

1.1 Installation

There are multiple sources for installing the latest stable version of the Ansys Sphinx Theme. These include public PyPI, Ansys PyPI, and GitHub.

Public PyPI

```
python -m pip install ansys-sphinx-theme
```

Ansys PyPI

```
PIP_EXTRA_INDEX_URL="https://$
↪{PYANSYS_PYPI_PRIVATE_PAT}@pkgs.dev.azure.com/pyansys/_packaging/pyansys/pypi/simple/
↪"
python -m pip install ansys-sphinx-theme
```

GitHub

```
python -m pip install git+https://github.com/ansys/ansys-sphinx-theme.git@main
```

CHAPTER

TWO

USER GUIDE

This section outlines the fundamental configurations of the Ansys Sphinx theme and how to integrate third-party extensions with this theme to customize your documentation.

Basic configuration Configure the Ansys Sphinx Theme

Basic configuration *HTML theme options* Set theme options

HTML theme options *The linkcode extension* See how to use the `linkcode` extension with the Ansys Sphinx Theme.

The linkcode extension *Sphinx AutoAPI* See how to use Sphinx AutoAPI with the Ansys Sphinx Theme.

Sphinx AutoAPI

2.1 Basic configuration

To use the Ansys Sphinx Theme, add the following line to your project's Sphinx `conf.py` file:

```
html_theme = "ansys_sphinx_theme"
```

The Ansys Sphinx Theme provides these features:

- *PyAnsys and Ansys logos*
- *Version switcher*
- *PDF cover page*
- *Custom CSS*

2.1.1 PyAnsys and Ansys logos

The Ansys Sphinx Theme includes the PyAnsys and Ansys logos. All the logos are available in the ansys_sphinx_theme/static/ directory.

To use the logo in both dark and light modes, add the following code to the``html_theme_options`` dictionary in your project's Sphinx `conf.py` file:

Ansys logo

```
html_theme_options = {
    "logo": "ansys",
}
```

PyAnsys logo

```
html_theme_options = {
    "logo": "pyansys",
}
```

No logo

```
html_theme_options = {
    "logo": "no_logo",
}
```

Note

By default, if `ansys` logo is displayed, the logo links to the Ansys website. If the PyAnsys logo is displayed, the logo links to the PyAnsys website. If you want to change the link, you can set the `logo_link` option in the `conf.py` file.

For example:

```
html_theme_options = {
    "logo": "ansys",
    "logo_link": "https://www.example.com",
}
```

Note

If you use the `logo` option, make sure to remove the `html_logo` option from the `conf.py` file. `logo` option overrides the `html_logo` option and display the specified logo.

You can also add a custom logo by specifying the path to the logo file as specified in pydata-sphinx-theme.

For example:

```
html_theme_options = {
    "logo": {
        "image_light": "_static/logo-light.png",
        "image_dark": "_static/logo-dark.png",
    }
}
```

2.1.2 `favicon`

The `favicon` setting specifies the icon that appears in the browser tab. To use the Ansys favicon, add the following code to your project's Sphinx `conf.py` file:

```
html_favicon = ansys_favicon
```

2.1.3 Version switcher

The Ansys Sphinx Theme includes a version switcher for switching between different versions of the documentation. To show the version switcher in your documentation, add the following code to your project's Sphinx `conf.py` file:

```
from ansys_sphinx_theme import get_version_match

version = "0.1.0"
switcher_versions = get_version_match(version)
cname = "your_name"
html_theme_options = {
    "switcher": {
        "json_url": f"https://{cname}/versions.json",
        "version_match": switcher_version,
    },
}
```

The switcher requires a `versions.json` file that contains the versions of the documentation and their URLs in the given `json_url`. For more information, see PyAnsys multi-version documentation in the *PyAnsys developer's guide*.

2.1.4 PyAnsys tags

The Ansys Sphinx Theme allows you to add a physics tag in your metadata based on the pyansys category. This can be useful for categorizing content related to specific physics domains, such as `Electromagnetics`, `Structures`, or `Fluids`. You can add a physics tag to your documentation by setting the `pyansys_tags` option in your project's Sphinx `conf.py` file.

```
html_context = {
    "pyansys_tags": ["Electromagnetics"],
}
```

which result in the following metadata in your HTML pages:

```
<meta property="og:site_name" content="PyAnsys" />
<meta name="physics" content="Electromagnetics" />
```

If you want to add multiple physics tags, you can do so by providing a list of tags:

```
html_context = {
    "pyansys_tags": ["Electromagnetics", "Structures", "Fluids"],
}
```

This result in multiple *<meta>* tags in your HTML pages:

```
<meta property="og:site_name" content="PyAnsys" />
<meta name="physics" content="Electromagnetics" />
<meta name="physics" content="Structures" />
<meta name="physics" content="Fluids" />
```

2.1.5 PDF cover page

The Ansys Sphinx Theme includes a PDF cover page that you can customize. To customize the PDF cover page, see *PDF cover page*.

2.1.6 Custom CSS

You can add custom CSS to the Ansys Sphinx Theme by creating a directory named `_static/css` in your documentation and adding the following code to your project's Sphinx `conf.py` file:

```
html_static_path = ["_static"]
html_css_files = ["css/custom.css"]
```

Here is an example of a custom CSS file that changes the background color of the body to black and the text color to white:

```
.body {
    background-color: black;
   color: white;
}
```

2.2 PDF cover page

For generating a PDF of your documentation, Sphinx uses a default cover page. However, you can use the `generate_preamble` function in the `ansys_sphinx_theme.latex` module to create and use a custom cover page:

`ansys_sphinx_theme.latex.`**`generate_preamble`**(*title*, *watermark='watermark'*, *date=None*)

Generate the preamble for the PDF documentation.

Parameters

title
[`str`] Title of the document.

watermark
[`str`, `optional`] Name of the watermark image.

date
[`datetime`, `optional`] Date of document generation. If not provided, today's date is used.

Returns

`str`
A string representing the LaTeX source code for the preamble.

You use this function to generate the value for the `preamble` key in the `latex_elements` variable declared in the `conf.py` file:

```
latex_elements = {
    "preamble": ansys_sphinx_theme.latex.generate_preamble(
        <title_of_coverpage>,
        <watermark_for_titlepage>,
        <date_to_be_printed>,
    )
}
```

To use the logo and watermark provided by Ansys on the cover page, you must import them and then add them to the `latex_additional_files` dictionary:

```
from ansys_sphinx_theme import (
    ansys_logo_white,
    ansys_logo_white_cropped,
```

(continues on next page)

(continued from previous page)

```
    watermark,
)
```

```
latex_additional_files = [watermark, ansys_logo_white, ansys_logo_white_cropped]
```

For an example of a rendered PDF cover page, see the PDF documentation.

2.3 HTML theme options

In the Sphinx configuration (`conf.py`) file in the `doc` directory, you can use the `html_theme_options` dictionary to customize the Ansys Sphinx theme.

2.3.1 Table of contents

Option	Type	Description
Show breadcrumbs	bool	Show breadcrumbs at the top of each page.
Add and hide icons in the navigation bar	list	Add and hide icons in the navigation bar.
Static search options	list	Additional static search options to customize the search experience.
Cheat sheets	list	Add a cheat sheet to the left navigation pane of your documentation.
What's new section	list	Show a "What's new" section in the left navigation pane.
News and resources section	dict	Show a filterable news and resources page and add a link to it in the left navigation pane.
Secondary sidebar	list	Control the secondary sidebar on each page.
Navigation bar dropdown	list	Add a dropdown navigation bar to the top of your documentation.
Announcement banner	list	Add an announcement banner to the top of your documentation pages.
MCP server banner	dict	Promote an MCP server on the landing page with a banner linking to its repository.

2.3.2 Show breadcrumbs

Showing breadcrumbs at the top of your documentation pages makes navigation easier. Breadcrumbs are shown by setting `"show_breadcrumbs": True`. To add additional *root* breadcrumbs, `"additional_breadcrumbs"` is set to a list of tuples in this form: `("Link text", "url")`.

This `html_theme_options` dictionary show breadcrumbs, including a root breadcrumb for the documentation landing page for the Ansys repository:

```
html_theme_options = {
    "github_url": "https://github.com/ansys/ansys-sphinx-theme",
    "show_prev_next": True,
    "show_breadcrumbs": True,
    "additional_breadcrumbs": [
        ("PyAnsys", "https://docs.pyansys.com/"),
    ],
}
```

When you are on the landing page for your documentation, the breadcrumb shows the title for this page. However, Sphinx cannot access this title from other documentation pages. Thus, after `html_theme_options` dictionary, you must set `html_short_title` to the display text to use for this breadcrumb.

To ensure a consistent user experience, always set the `html_short_title` (or optionally `html_title` if `html_short_title` is not used) to the library name.

For example, in the `conf.py` file for the Ansys Sphinx Theme, this line is added after the `html_theme_options` dictionary:

```
html_short_title = html_title = "Ansys Sphinx Theme"
```

If you want the title for your documentation's main `index.rst` file to show the version, include `|version|` in the title:

```
html_short_title = html_title = "Ansys Sphinx Theme |version|"
```

2.3.3 Add and hide icons in the navigation bar

The navigation bar shows two icons on the right by default. The first is for switching between light and dark modes. The second is for going to the library's GitHub repository.

- For comprehensive information on adding custom link behavior, see Add custom attributes to icon links in the PyData Theme documentation.
- For comprehensive information on how to use Font Awesome to add icons, see How To Add Icons in the Font Awesome documentation.

The following sections explain how to add icons and hide icons.

Add icons

In the `conf.py` file, the `html_theme_options` dictionary has a child `icon_links` dictionary. To add icons to the navigation bar, add them to the `icon_links` dictionary. For each icon to add, specify its `name`, the associated `url`, the `icon`, and the `type`.

This example adds an icon for sending an email:

```
html_theme_options = {
 "icon_links": [
     dict(name="Mail", url="mailto:me", icon="fas fa-envelope")
 ],
 ...
}
```

Hide icons

To hide icons so that they do not show in the navigation bar, add their names to the `hidden_icons` dictionary:

```
html_theme_options = {
    "hidden_icons": ["GitHub"],
    ...
}
```

If you want to hide all icons, use the `show_icons` Boolean variable:

```
html_theme_options = {
    "show_icons": False,
    ...
}
```

2.3.4 Static search options

The Ansys Sphinx theme supports static search options to customize the search experience.

The static search bar is created using `Fuse.js`. You can provide all options supported by `Fuse.js` through the `static_search` dictionary in the `html_theme_options`.

To improve the search performance, a debounce function is available. By default, a delay of 300 milliseconds is applied. To modify this value, declare the `delay` key in the `static_search` dictionary with a value specifying the amount of milliseconds to wait before executing the search. A value of `0` disables the debounce function.

Additionally, you can decide the limit of the search results by setting the `limit` key in the `static_search` dictionary. The default value is `10`.

To exclude files or directories from the search index, you can use the `files_to_exclude` key in the `static_search` dictionary. This key is a list of strings representing the directories or files to exclude from the search index.

Search result ranking

By default, the theme assigns the following Fuse.js field weights so that section headings and page titles rank over the body text:

Field	Default weight	Description
`section`	3	Section heading extracted from the page.
`title`	3	Full breadcrumb title of the page.
`text`	1	Body text of the section.
`objectID`	0.5	Internal document identifier.

You can override the weights by supplying a `keys` list in `static_search`:

```
html_theme_options = {
    "static_search": {
        "keys": [
            {"name": "section", "weight": 5},
            {"name": "title", "weight": 3},
            {"name": "text", "weight": 1},
            {"name": "objectID", "weight": 0.5},
        ],
    },
}
```

The `includeScore` and `includeMatches` options are enabled by default so that match highlighting and relevance sorting work out of the box. You can turn off these options if you do not need that information:

```
html_theme_options = {
    "static_search": {
        "includeScore": False,
        "includeMatches": False,
    },
}
```

Here is a full example combining the most commonly used options:

```
html_theme_options = {
    "static_search": {
        "threshold": 0.5,
        "limit": 10,
        "minMatchCharLength": 1,
        "delay": 300,
        "files_to_exclude": ["_build", "api/", "examples/sphinx_demo"],
    },
}
```

Note

All other options are available in the Fuse.js documentation. Any key you set takes precedence over the theme defaults; values you omit are filled in automatically.

Note

Serve locally your documentation using the `python -m http.server -d /path/to/docs/html/` to have a live-preview of your search results. This method is compliant with the CORS policy and allows to load the generated resource files containing the indices of your documentation. The search bar does not work if you open the HTML files directly in the browser.

To open the documentation in a local server, run the following command in the directory where the HTML files are located:

```
python -m http.server
```

Then, open the browser and go to `http://localhost:8000`.

Advanced search options

The Ansys Sphinx theme supports advanced search capabilities to enhance the user experience.

These options can be configured through the `html_theme_options` dictionary in your `conf.py` file.

Multi-index search

To enable search across multiple documentation sources, use the `search_extra_sources` key. This key should be a dictionary where each key is the name of the source and the value is the URL to that source.

Example:

```
html_theme_options = {
    "search_extra_sources": {
        "PyMAPDL": "https://mapdl.docs.pyansys.com/version/stable/",
        "PyAnsys": "https://docs.pyansys.com/version/stable/",
    },
}
```

Search filters

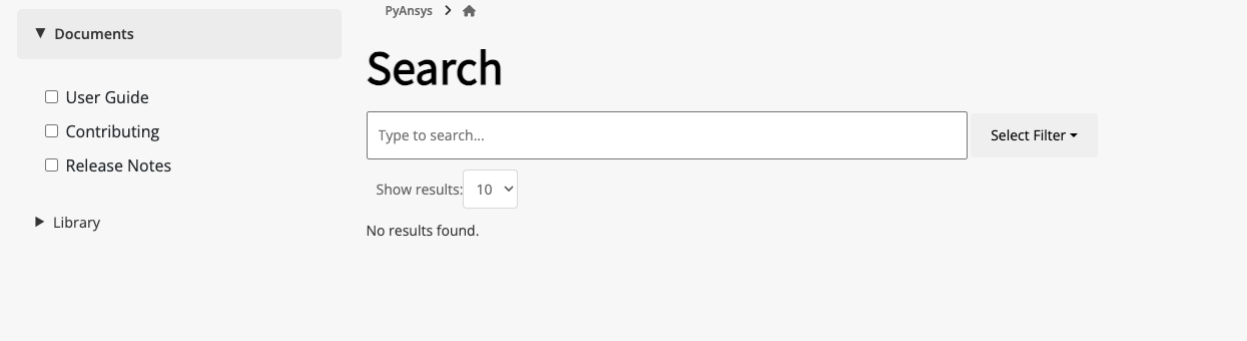
To organize and group search results, you can define custom filters using the `search_filters` key. This key should be a dictionary where each key represents a filter label and the corresponding value is a list of directories or file paths that belong to that filter.

Example:

```
html_theme_options = {
    "search_filters": {
        "User Guide": [
            "user-guide/",
            "getting-started/",
            "index/",
        ],
        "Release Notes": ["changelog"],
        "Examples": ["examples/"],
        "Contributing": ["contribute/"],
    },
}
```

The filters appears as clickable options in the search interface, allowing users to refine their results by content type.

The search filters are displayed as below:



2.3.5 Cheat sheets

If a cheat sheet has been created for your PyAnsys library, with `quarto`, you can add it to the left navigation pane of your documentation.

In the `html_theme_options` dictionary, you add a child dictionary named `cheatsheet` that contain these keys, in the order given:

1. `file`: filename including the extension of the cheat sheet. If the file is inside a directory, include the directory name relative to the root of the documentation. For example, if the cheat sheet is in the `getting_started` directory, the filename is `getting_started/cheat_sheet.qmd`.
2. `title`: Title of the cheat sheet to be displayed in the left navigation pane.
3. `pages`: List of names for the pages to include the cheat sheet on. If no value is provided, the cheat sheet is displayed only on the main `index.html` file.
4. `version`: Version of the cheat sheet. If no value is provided, the version is `main` by default.

Here is an example of how to add the `cheatsheet` dictionary to the *html_theme_options`* dictionary:

```
html_theme_options = (
    {
        "cheatsheet": {
            "file": "<filename including the extension of the cheat sheet>",
            "title": "<title of the cheat sheet>",
            "version": "<version of the cheat sheet>",
            "pages": "<list of names for the pages to include the cheat sheet on>",
↪# Optional
        },
    },
)
```

Here is an example of how to show a thumbnail of a PyMAPDL cheat sheet in the left navigation pane of its main `index.rst` file and the `learning.rst` file in its "Getting started" section:

```
html_theme_options = (
    {
        "cheatsheet": {
            "file": "getting_started/cheat_sheet.qmd",
            "title": "My awesome cheat sheet",
            "version": f"{version}",
            "pages": ["index", "getting_started/learning"],
        },
    },
)
```

Note

To use this feature, you must have the *quarto <https://quarto.org/>* package installed. To create thumbnails of generated PDF files, the theme is using *pdf2image*. So you should have the `poppler` package installed in your system. For more information, see the pdf2image documentation.

2.3.6 What's new section

The "What's new" section is an option that allows you to highlight new features in your library for each minor version within the changelog file.

To install the "What's new" dependencies from ansys-sphinx-theme, run the following command:

```
pip install ansys-sphinx-theme[changelog]
```

To get started, create a YAML file named `whatsnew.yml` in the `doc/changelog.d` directory. The YAML file should contain the following structure:

```
fragments:
- title: Feature title
  version: 0.2.0  # The version the feature is introduced
  content: |
    Feature description in RST format.

- title: Another feature title
  version: 0.1.2
```

(continues on next page)

(continued from previous page)

```
  content: |
    Feature description in RST format.
```

The dropdown generation only supports the following RST formats in the "content" field:

- Bold: Use double asterisks to wrap the text.
- Italics: Use single asterisks to wrap the text.
- Code samples: Use single or double backticks to wrap the text.
- Links: Use the following format to include links:

  ```
  `link text <https://www.example.com>`_
  ```

- Code blocks: Use the following format to include code blocks:

  ```
  .. code:: python

     print("hello world")
  ```

If a format is used in the `content` field that does not fall into the categories above, it is not be rendered correctly.

To enable the **What's new** section and sidebar in the changelog file, add the following dictionary to the `html_theme_options` dictionary:

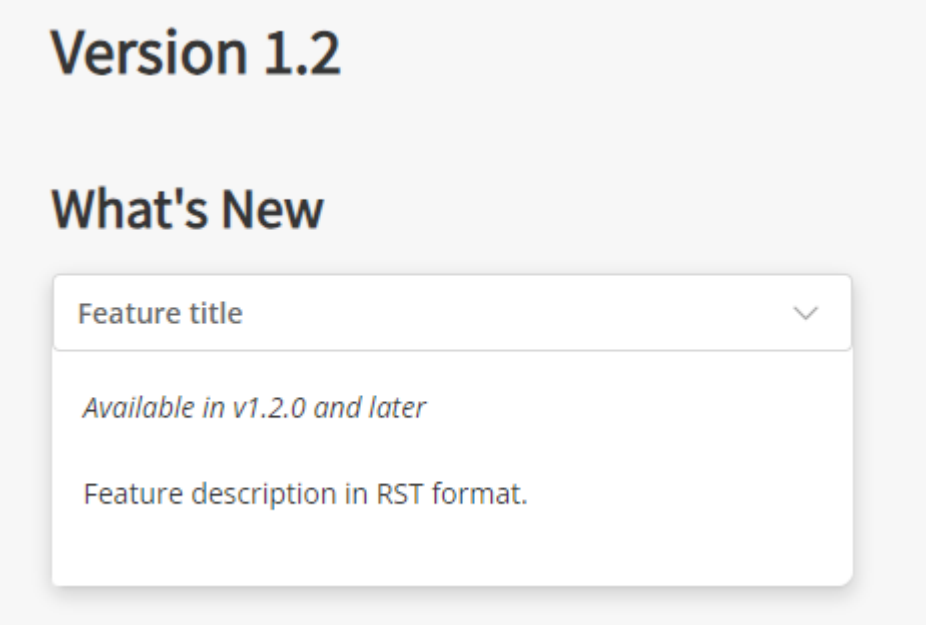
```
html_theme_options = (
    {
        "whatsnew": {
            "whatsnew_file_path": "../changelog.d/whatsnew.yml",
            "changelog_file_path": "changelog.rst",
            "sidebar_pages": ["changelog"],
            "sidebar_no_of_headers": 3,  # Optional
            "sidebar_no_of_contents": 3,  # Optional
        },
    },
)
```

The dictionary contains the following keys:

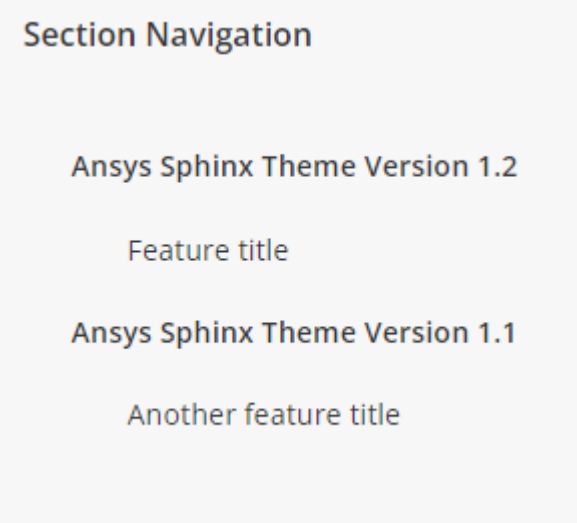
- `whatsnew_file_path`: Path to the YAML file containing what's new content local to the `doc/source` directory. If not provided, the what's new section is not generated.
- `changelog_file_path`: Path to the `changelog.rst` file local to the `doc/source` directory. If not provided, the **What's new** section is not generated.
- `sidebar_pages`: List of names for the pages to include in the **What's new** sidebar. If not provided, the **What's new** sidebar is not shown.
- **`sidebar_no_of_headers`: Number of minor version sections to display in the What's new sidebar.** By default, it displays three version sections in the sidebar.
- **`sidebar_no_of_contents`: Number of what's new content to display under each minor version in the** what's new sidebar. If not provided, it displays all dropdowns by default.

The following images show a sample "What's new" section and sidebar in the changelog file:

What's new section



What's new sidebar



Note

If you are displaying both the **What's new** and **Cheat sheet** sections, the **Cheat sheet** section is shown first in the left navigation pane, followed by the **What's new** section to maintain sidebar consistency.

2.3.7 News and resources section

Use the "News and resources" section to publish a filterable list of videos, blog posts, events, and other news on a dedicated page, with a link to it in the left navigation pane.

Define entries

Define each entry with the `news-item` directive. The directive body is the description of the entry. It is rendered as plain text, so RST markup is not interpreted.

```
.. news-item::
    :type: Video
    :title: PyAnsys 2025 R1 walkthrough
    :author: PyAnsys Team
    :date: 2025-01-15
    :link: https://example.com/video

    Full support for the new solver API and improved mesh handling.
```

The directive supports the following options:

Option	Required	Description
`type`	Yes	Category of the entry. Any value is accepted and is matched case-insensitively. `Video`, `Blog`, `Event`, and `News` have their own badge color. Any other value, such as `Webinar`, uses a neutral badge. See *Style a custom type* to set the color of a custom type.
`title`	Yes	Title of the entry.
`author`	Yes	Author name.
`date`	Optional	Free-form string, such as `2025-01-15`. Entries with a date are sorted by this value, newest first, and listed before entries without a date. Entries with the same date, and entries without a date, keep the order of the RST file. Use a sortable format such as ISO 8601.
`link`	Optional	URL that the title links to. Only `http` and `https` URLs are rendered as links. Other schemes are ignored and a warning is emitted.

Display the entries

Entries are collected per page and rendered as a card list on the same page where they are defined. The list includes **Type** and **Author** filters and is paginated with 10 entries per page.

Place the `news-resources-table` directive where the list should appear:

```
.. news-resources-table::
```

If the directive is omitted, the list is appended automatically to the end of any page that contains `news-item` directives. Because each page only shows its own entries, define all entries on the page that you want to show as the news and resources page.

Style a custom type

To set the badge color of a type, including a custom one, add a `badge_colors` dictionary to the `news_resources` option in `conf.py`. Keys are types (case-insensitive) and values are CSS colors (hex, named, or `rgb()`/`hsl()`):

```
html_theme_options = {
    "news_resources": {
        "link": "news_resources",
        "badge_colors": {
            "webinar": "#d8c5f0",
            "video": "#ffb347",  # overrides the default color
```

(continues on next page)

(continued from previous page)

```
        },
    },
}
```

Invalid colors are ignored and a warning is emitted.

Enable the sidebar link

To add the link to the left navigation pane, add the following dictionary to `html_theme_options`:

```
html_theme_options = {
    "news_resources": {
        "link": "news_resources",
        "pages": ["index"],
        "title": "News & Resources",
    },
}
```

The dictionary contains the following keys:

Key	Required	Description
`link`	Yes	Document name, without the extension and relative to the `doc/source` directory, of the page that contains the entries. For example, `news_resources` for `news_resources.rst`. If it is missing, a warning is emitted and the link is not valid.
`pages`	Optional	List of document names (or a single name) where the link is shown. The default is `["index"]`.
`title`	Optional	Title of the sidebar link. The default is `"News & Resources"`.
`badge_colors`	Optional	Dictionary mapping an entry type to a badge color. See *Style a custom type*.

The link is shown in the overview sidebar, which also contains the package home link, and the **Cheat sheet** and **What's new** entries when they are configured. This sidebar is always shown on the `index` page and on the page set in `link`.

Note

For non-HTML builders, such as LaTeX, the `news-item` and `news-resources-table` directives are still parsed, but the page set in `link` is excluded from the build (as `<link>.rst`).

2.3.8 Navigation bar dropdown

This theme supports dropdown navigation bars. The layout is declared using a YAML file contained at any level in the `doc/source` directory. The file path is relative to the `doc/source` directory,and must be specified in the `html_theme_options` dictionary.

- `navigation_yaml_file`: The path to the YAML file containing the navigation structure.

```
html_theme_options = {
    ...,
    "navigation_dropdown": {
        "layout_file": "navbar.yml", # Relative path to the YAML file
```

(continues on next page)

(continued from previous page)

```
    },
}
```

Each entry in the YAML file may include the following fields:

- **file.** The relative path to the documentation file, based on the doc/source directory.
- **title.** The text displayed for the link in the dropdown navigation menu.
- **sections.** A list of nested navigation items. Each section can specify its own file, title, and an optional caption to provide a brief description.

```
- file: api/index
  title: "API Reference"

- file: examples
  title: "Examples"
  sections:

    - file: examples/sphinx-design.rst
      title: "Sphinx Design Examples"
      caption: Examples of using Sphinx design features

    - file: examples/nbsphinx
      title: "Nbsphinx Examples"
      caption: Examples of using Nbsphinx for Jupyter Notebooks
```

Warning

You must declare the complete layout of the dropdown navigation bar in the YAML file. Sphinx does not resolve it automatically.

2.3.9 Secondary sidebar

The right-hand secondary sidebar shows the **"On this page"** table of contents (TOC), which lists the section anchors for the current page. It also contains the **Edit this page** and **View source** links.

Hide the page TOC globally

Set `"show_page_toc": False` to hide the page TOC on every page. The **Edit this page** and **View source** links remain visible:

```
html_theme_options = {
    "show_page_toc": False,
}
```

Show the page TOC inside the primary sidebar

When the secondary sidebar TOC is turned off by with `"show_page_toc": False`, you can move the TOC into the primary (left-hand) sidebar instead by setting `"show_page_toc_in_primary_sidebar": True`.

```
html_theme_options = {
    "show_page_toc": False,
    "show_page_toc_in_primary_sidebar": True,
}
```

The TOC entries are injected directly under the current page's entry in the section navigation tree. A small chevron toggle (›) appears beside the page link. Clicking it expands or collapses the entries manually. As you scroll through the page, the TOC auto-expands and highlights the active section heading automatically.

Hide the View source button globally

Set `"show_source_button": False` to remove the **View source*** link from every page. The page TOC and **Edit this page** link remain visible:

```
html_theme_options = {
    "show_source_button": False,
}
```

Both options can be combined:

```
html_theme_options = {
    "show_page_toc": False,
    "show_source_button": False,
}
```

Control the secondary sidebar per page

For finer control, use the `secondary_sidebar_items` option from pydata-sphinx-theme. You can pass a list of item names (applied globally) or a dictionary with glob patterns as keys (applied per page).

```
html_theme_options = {
    # Remove the page TOC from all pages but keep other items.
    "secondary_sidebar_items": {
        "**": ["edit-this-page", "sourcelink"],
    },
}
```

```
html_theme_options = {
    # Show the page TOC on regular pages; hide it on API and index pages.
    "secondary_sidebar_items": {
        "**": ["page-toc", "edit-this-page", "sourcelink"],
        "api/**": ["edit-this-page", "sourcelink"],
        "index": [],
    },
}
```

Control the TOC depth

Use `show_toc_level` to set how many heading levels are expanded by default in the page TOC. The default is `1`, which causes only top-level headings to be shown. This code would show h2 and h3 headings by default:

```
html_theme_options = {
    "show_toc_level": 2,  # Expand h2 and h3 entries by default
}
```

Hide the page TOC on a single page

To suppress the entire secondary sidebar for a single RST page without changing the `conf.py` file, add the following metadata at the very top of the file:

```
:html_theme.sidebar_secondary.remove:

Page title
==========
...
```

2.3.10 Announcement banner

The Ansys Sphinx Theme supports announcement banners that can be displayed at the top of documentation pages. These banners can be used to communicate important information, such as product updates, deprecation notices, maintenance alerts, or other announcements.

To configure an announcement banner, use the `announcement_banner` option in the `html_theme_options` dictionary of your `conf.py` file. This option accepts a list of announcement dictionaries, as shown in the following example:

```
html_theme_options = {
    "announcement_banner": [
        {
            "message": "Welcome to the Ansys Sphinx Theme documentation!",
            "type": "info",
        },
        {
            "message": "This release is deprecated. Please upgrade.",
            "type": "warning",
            "link": "https://docs.pyansys.com/version/stable/",
        },
        {
            "message": "Critical security patch required.",
            "type": "error",
        },
        {
            "message": "New features available in the latest release.",
            "type": "success",
        },
    ],
}
```

Each announcement supports the following keys:

- `message`: Text displayed in the announcement banner.
- `type`: Type of announcement. Supported values are:
  - `info`: Informational messages.
  - `success`: Successful operations or positive updates.

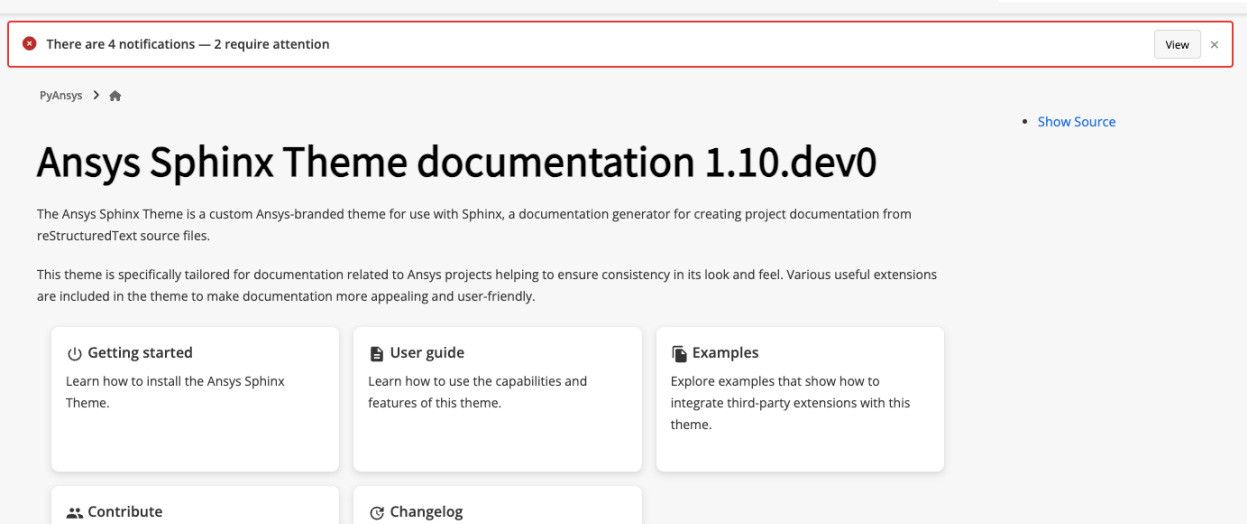
  - `warning`: Warnings or deprecation notices.
  - `error`: Critical alerts or issues requiring immediate attention.

- `link` *(optional)*: URL associated with the announcement. When provided, the banner message becomes clickable and directs users to the specified URL for additional information.

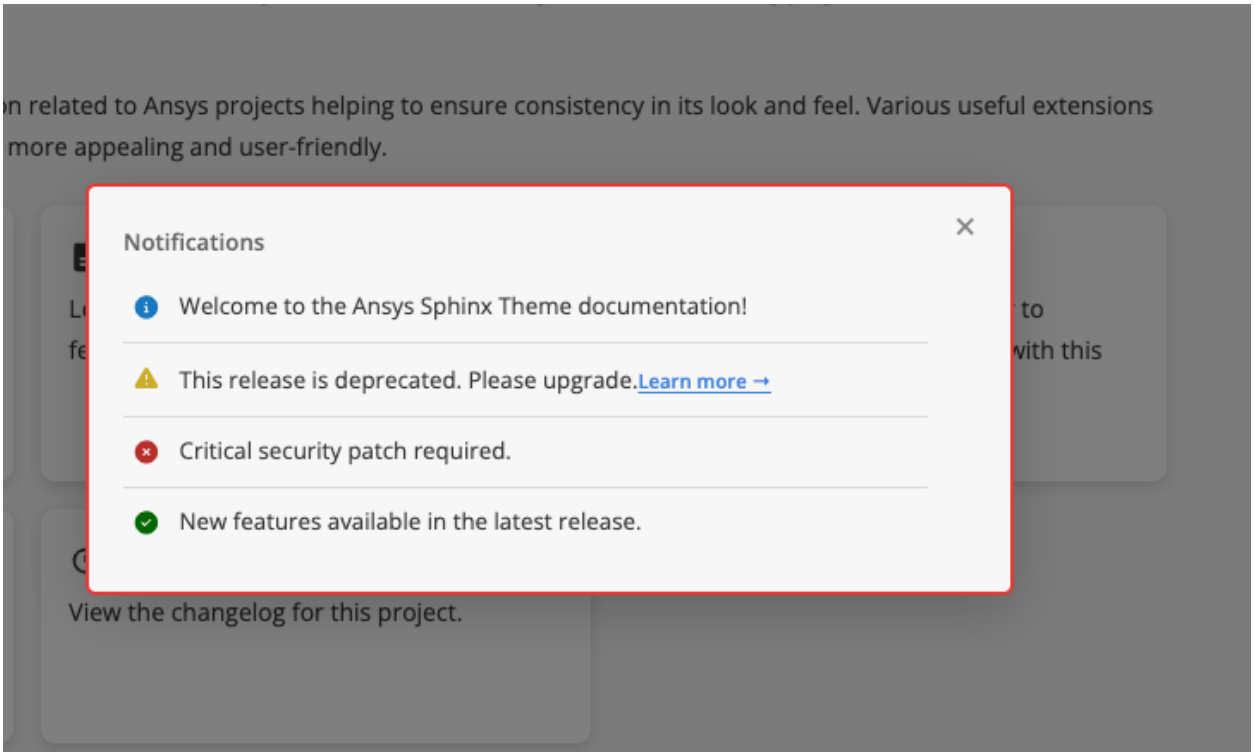
Announcement banners follow this priority order: **Error > Warning > Success > Info**. When multiple announcement types are configured, the banner adopts the highest-priority type. For instance, if both Error and Warning announcements are available, the landing page banner displays with the Error styling.

The order of the announcements in the list determines their display order, with the first announcement appearing at the top of the banner.

For example, for the configuration above, below is how the announcement banner appears in the documentation:



When the user clicks on the `view` button, the banner expands to show all announcements in the order they are defined in the configuration.



2.3.11 MCP server banner

Many Ansys libraries provide a companion Model Context Protocol (MCP) server that exposes library functionality to AI assistants and tools. To promote the MCP server from your library's documentation landing page, add the `mcp_server` dictionary to `html_theme_options` in your `conf.py` file:

```
html_theme_options = {
    ...,
    "mcp_server": {
        "url": "https://github.com/ansys/pymechanical-mcp",
        "project_name": "PyMechanical MCP Server",
    },
}
```

The dictionary supports the following keys:

- `url` *(required)*: URL pointing to the MCP server repository or documentation.
- `project_name` *(optional)*: Human-readable name for the MCP server. Defaults to `"MCP Server"` if not provided.

When configured, a banner is displayed at the top of the documentation landing page (`index.html`). The banner includes:

- A brief description inviting users to connect their AI assistant to the library.
- A **Learn more** link that opens the configured `url` in a new tab.

The banner is shown **only on the landing page** and does not appear on any other page.

Example configuration for `PyMechanical`:

```
html_theme_options = {
    "github_url": "https://github.com/ansys/pymechanical",
    "mcp_server": {
        "url": "https://github.com/ansys/pymechanical-mcp",
        "project_name": "PyMechanical MCP Server",
    },
}
```

2.4 The `linkcode` extension

The `linkcode` extension automatically adds *source* links to the documentation for Python, C, C++, and JavaScript objects. It allows you to link to the source code hosted on GitHub.

To use the `linkcode` extension, you must add it to the `extensions` variable in your project's Sphinx `conf.py` file:

```
extensions = ["ansys_sphinx_theme.extension.linkcode"]
```

2.4.1 Configuration options

The Linkcode extension provides a way to configure its behavior by using certain options within your `conf.py` file. Depending on your preferred approach, you can utilize the direct configuration options or the `html_context` dictionary to streamline your settings.

If both sets of configuration options are given, the direct configuration options (that is, `link_code_library`, `link_code_source`, `link_code_branch`) has precedence over the corresponding settings in the `html_context` dictionary.

Direct configuration options

- `link_code_library` : The user/repository name where the source code is hosted. For example, `ansys/ansys-sphinx-theme`.
- `link_code_source` (str, optional, default: ''): The relative path of the source code file within the repository. For example, `src`.
- `link_code_branch` (str, optional, default: 'main'): The GitHub branch. It can be a specific version like `main` or `dev`.

If the `link_code_source` and `link_code_branch` options are not provided in the configuration, the following default values are used:

- `link_code_source`: An empty string (`''`). This links to the root of the repository.
- `link_code_branch`: `main`. This is the default branch name used if no branch is specified.

```
# Example of setting direct configuration in example
link_code_library = "username/repo-name"
link_code_source = "src"
link_code_branch = "dev"
```

Using `html_context` dictionary

You also have the option to centralize your GitHub-related configuration by incorporating it directly into the `html_context` dictionary within your `conf.py` file. This approach allows you to minimize redundancy and manage the GitHub-related information more effectively for the extension:

```
# Example of setting GitHub-related configuration in conf.py
html_context = {
    "github_user": "<your-github-org>",
    "github_repo": "<your-github-repo>",
    "github_version": "<your-branch>",
    "source_path": "<path-from-root-to-your-source_file>",
}
```

With this setup, you can fine-tune your configuration according to your preferences and requirements, enhancing the integration of the `linkcode` extension into your documentation.

2.5 Sphinx AutoAPI

To use Sphinx AutoAPI with the Ansys Sphinx Theme, you must add `ansys_sphinx_theme.extension.autoapi` to the `extensions` list in your `conf.py` file and set the `ansys_sphinx_theme_autoapi` theme options in the `html_theme_options` dictionary.

- `project`: The name of the project.
- `output`: The path to the directory where the generated files are placed. By default, this is set to the `api` directory.
- `templates`: The path to the directory containing the custom templates for `sphinx-autoapi`. By default, this is set to the `autoapi_templates` directory in the theme package.
- `directory`: The path to the directory containing the source code with respect to the `conf.py` file. By default, this is set to the `src/ansys` directory.
- `use_implicit_namespaces`: If set to `True`, the autoapi extension use *implicit namespaces*. By default, this is set to `True`.
- `keep_files`: If set to `True`, the autoapi extension keeps the generated files. By default, this is set to `True`.
- `own_page_level`: The level of the page where the autoapi extension places the content of the class. By default, this is set to `class`.
- `type`: The type of the autoapi extension. By default, this is set to `python`.
- `options`: The options to pass to the autoapi extension. By default, this is set to `["members", "undoc-members", "show-inheritance", "show-module-summary", "special-members"]`.
- `class_content`: The content of the class. By default this is set to `class`.
- `ignore`: The list of directories to ignore. By default, this is empty.
- `add_toctree_entry`: If set to `True`, the autoapi extension adds the generated files to the TOC tree. By default, this is set to `False`.
- `package_depth`: The depth of the package. By default, this is set to `3`. This is the `namespace` depth of the package. For example, if the package is `ansys`, the depth is `1`. If the package is `ansys.foo`, the depth is `2`.
- `member_order`: The order to document members. By default, this is set to `bysource`. Other options include `alphabetical`, which orders members by their name (case sensitive), or `groupwise`, which orders members by their type and alphabetically.

All these options can be set in the `conf.py` file of your Sphinx project.

```
html_theme_options = {
    "ansys_sphinx_theme_autoapi": {
        "project": "My Project",
```

(continues on next page)

(continued from previous page)

```
        "output": "api",
        "directory": "src/ansys",
        "use_implicit_namespaces": True,
        "keep_files": True,
        "own_page_level": "class",
        "type": "python",
        "options": [
            "members",
            "undoc-members",
            "show-inheritance",
            "show-module-summary",
            "special-members",
        ],
        "class_content": "class",
        "ignore": [],
        "add_toctree_entry": False,
        "package_depth": 3,
        "member_order": "bysource",
    }
}
```

You need to add `ansys_sphinx_theme.extension.autoapi` to the `extensions` list in your `conf.py` file:

```
extensions = [
    "ansys_sphinx_theme.extension.autoapi",
]
```

The complete configuration for `sphinx-autoapi` in your `conf.py` file should look like this:

```
html_theme_options = {
    "ansys_sphinx_theme_autoapi": {
        "project": "My Project",
        "output": "api",
        "use_implicit_namespaces": True,
        "directory": "src/ansys",
        "keep_files": True,
        "own_page_level": "class",
        "type": "python",
        "options": [
            "members",
            "undoc-members",
            "show-inheritance",
            "show-module-summary",
            "special-members",
        ],
        "class_content": "class",
    }
}

extensions = [
    "ansys_sphinx_theme.extension.autoapi",
]
```

CHAPTER

THREE

EXAMPLES

This section provides examples that show how the Ansys Sphinx Theme renders different Sphinx extensions and documentation components.

3.1 Sphinx extensions

These examples show how the Ansys Sphinx Theme renders different Sphinx extensions.

Sphinx design Examples of how the Sphinx design is rendered.

Sphinx design Jupyter notebook - nbsphinx Examples of how notebooks are rendered.

Jupyter notebook - nbsphinx Sphinx gallery Examples of how the Sphinx gallery is rendered.

Sphinx-Gallery Sphinx autoapi Examples of how Sphinx AutoAPI documentation is rendered.

Sphinx AutoAPI

3.2 Documentation components

These examples show how the Ansys Sphinx Theme renders different documentation components.

Tables Examples of tables with JavaScript and RST rendered using the *ansys-sphinx-theme*.

Table Admonitions Examples of how admonitions are rendered.

Admonitions

3.2.1 Sphinx design

This example shows how the Ansys Sphinx Theme renders documentation components using the `sphinx-design` extension. For comprehensive information, see its documentation.

Credits and acknowledgments

The examples presented below are sourced from the Sphinx Design project Copyright (c) by Chris Sewell (2023), available at sphinx design examples. This project is licensed under the MIT License.

Badge-basic

```
:bdg:`plain badge`

:bdg-primary:`primary`, :bdg-primary-line:`primary-line`
```

(continues on next page)

(continued from previous page)

```
:bdg-secondary:`secondary`, :bdg-secondary-line:`secondary-line`

:bdg-success:`success`, :bdg-success-line:`success-line`

:bdg-info:`info`, :bdg-info-line:`info-line`

:bdg-warning:`warning`, :bdg-warning-line:`warning-line`

:bdg-danger:`danger`, :bdg-danger-line:`danger-line`

:bdg-light:`light`, :bdg-light-line:`light-line`

:bdg-muted:`muted`, :bdg-muted-line:`muted-line`

:bdg-dark:`dark`, :bdg-dark-line:`dark-line`

:bdg-white:`white`, :bdg-white-line:`white-line`

:bdg-black:`black`, :bdg-black-line:`black-line`
```

This directive renders as follows:

plain badge

primary, primary-line

secondary, secondary-line

success, success-line

info, info-line

warning, warning-line

danger, danger-line

light, light-line

muted, muted-line

dark, dark-line

white, white-line

black, black-line

Badge-link

```
:bdg-link-primary:`https://example.com`

:bdg-link-primary-line:`explicit title <https://example.com>`
```

This directive renders as follows:

https://example.com

explicit title

Button-link

```
.. button-link:: https://example.com

.. button-link:: https://example.com

    Button text

.. button-link:: https://example.com
    :color: primary

    Button text with options

.. button-link:: https://example.com
    :color: primary
    :shadow:

.. button-link:: https://example.com
    :color: primary
    :outline:

.. button-link:: https://example.com
    :color: secondary
    :expand:
```

This directive renders as follows:

https://example.com

Button text

Button text with options

https://example.com

https://example.com

https://example.com

Card-basic

```
.. card:: Card Title

    Card content
```

This directive renders as follows:

Card Title Card content

Card-carousel

```
.. card-carousel:: 2

    .. card:: card 1

        content
```

(continues on next page)

(continued from previous page)

```
    .. card:: card 2

        Longer

        content

    .. card:: card 3

    .. card:: card 4

    .. card:: card 5

    .. card:: card 6
```

This directive renders as follows:

card 1 content

card 2 Longer

content

card 3 card 4 card 5 card 6

Card-head-foot

```
.. card:: Card Title

    Header
    ^^^
    Card content
    +++
    Footer
```

This directive renders as follows:

Header

Card Title Card content

Footer

Card-images

```
.. grid:: 2 3 3 4

    .. grid-item::

        .. card:: Title
            :img-background: images/particle_background.jpg
            :class-card: sd-text-black
            :img-alt: my text

            Text
```

(continues on next page)

(continued from previous page)

```
.. grid-item-card:: Title
    :img-top: images/particle_background.jpg
    :img-alt:

    Header
    ^^^
    Content
    +++
    Footer

.. grid-item-card:: Title
    :img-bottom: images/particle_background.jpg

    Header
    ^^^
    Content
    +++
    Footer
```

This directive renders as follows:



Title Text



Header

Title Content

Footer

Header

Title Content

Footer



Card-link

```
.. _cards-clickable:

Clickable cards
...............

Using the ``link`` and ``link-type`` options, you can turn an entire card into a␣
↪clickable link. Try hovering over then clicking on the cards below:

.. card:: Clickable Card (external)
    :link: https://example.com

    The entire card can be clicked to navigate to https://example.com.

.. card:: Clickable Card (external)
    :link: https://example.com
    :link-alt: example.com

    The entire card can be clicked to navigate to https://example.com.

.. card:: Clickable Card (internal)
    :link: cards-clickable
    :link-type: ref

    The entire card can be clicked to navigate to the ``cards-clickable`` reference␣
↪card-link.txt.

.. card:: Clickable Card (internal)
    :link: cards-clickable
    :link-type: ref
    :link-alt: clickable cards

    The entire card can be clicked to navigate to the ``cards-clickable`` reference␣
↪card-link.txt.
```

This directive renders as follows:

Clickable cards

Using the `link` and `link-type` options, you can turn an entire card into a clickable link. Try hovering over then clicking on the cards below:

Clickable Card (external) The entire card can be clicked to navigate to https://example.com.

https://example.com Clickable Card (external) The entire card can be clicked to navigate to https://example.com.

example.com Clickable Card (internal) The entire card can be clicked to navigate to the `cards-clickable` reference card-link.txt.

Clickable cards Clickable Card (internal) The entire card can be clicked to navigate to the `cards-clickable` reference card-link.txt.

clickable cards

Card-title-link

```
.. _card-title-link.txt:
.. card:: Card Title https://example.com :ref:`link <card-title-link.txt>`

    Card content
```

This directive renders as follows:

Card Title https://example.com *link* Card content

Div-basic

```
.. div:: sd-text-center sd-font-italic sd-text-primary

    Some CSS styled text
```

This directive renders as follows:

Some CSS styled text

Dropdown-basic

```
.. dropdown::

    Dropdown content

.. dropdown:: Dropdown title

    Dropdown content

.. dropdown:: Open dropdown
    :open:

    Dropdown content
```

This directive renders as follows:

Dropdown content

Dropdown title

Dropdown content

Open dropdown

Dropdown content

Dropdown-options

```
.. dropdown:: Title
    :name: dropdown-options.txt
    :color: info
    :icon: alert
```

(continues on next page)

(continued from previous page)

```
    :margin: 1
    :class-container: class-container
    :class-title: class-title
    :class-body: class-body

    Dropdown content

:ref:`dropdown-options.txt`, :ref:`text <dropdown-options.txt>`
```

This directive renders as follows:

Title

Dropdown content

Title, *text*

Grid-basic

```
.. grid:: 1 2 3 4
    :outline:

    .. grid-item::

        A

    .. grid-item::

        B

    .. grid-item::

        C

    .. grid-item::

        D
```

This directive renders as follows:

A

B

C

D

Grid-card-columns

```
.. grid:: 2

    .. grid-item-card::
        :columns: auto
```

(continues on next page)

(continued from previous page)

```
        A

    .. grid-item-card::
        :columns: 12 6 6 6

        B

    .. grid-item-card::
        :columns: 12

        C
```

This directive renders as follows:

A

B

C

Grid-card

```
.. grid:: 2

    .. grid-item-card::  Title 1

        A

    .. grid-item-card::  Title 2

        B
```

This directive renders as follows:

Title 1 A

Title 2 B

Grid-gutter

```
.. grid:: 2
    :gutter: 1

    .. grid-item-card::

        A

    .. grid-item-card::

        B

.. grid:: 2
    :gutter: 3 3 4 5
```

(continues on next page)

(continued from previous page)

```
    .. grid-item-card::

        A

    .. grid-item-card::

        B
```

This directive renders as follows:

A

B

A

B

Grid-nested

```
.. grid:: 1 1 2 2
    :gutter: 1

    .. grid-item::

        .. grid:: 1 1 1 1
            :gutter: 1

            .. grid-item-card:: Item 1.1

                Multi-line

                content

            .. grid-item-card:: Item 1.2

                Content

    .. grid-item::

        .. grid:: 1 1 1 1
            :gutter: 1

            .. grid-item-card:: Item 2.1

                Content

            .. grid-item-card:: Item 2.2

                Content

            .. grid-item-card:: Item 2.3
```

(continues on next page)

(continued from previous page)

```
                Content
```

This directive renders as follows:

Item 1.1 Multi-line

content

Item 1.2 Content

Item 2.1 Content

Item 2.2 Content

Item 2.3 Content

Icon-fontawesome

```
An icon :fas:`spinner;sd-bg-primary sd-bg-text-primary`, some more text.
```

This directive renders as follows:

An icon , some more text.

Icon-material-design

```
- A regular icon: :material-regular:`data_exploration;2em`, some more text
- A coloured regular icon: :material-regular:`settings;3em;sd-text-success`, some␣
↪more text.
- A coloured outline icon: :material-outlined:`settings;3em;sd-text-success`, some␣
↪more text.
- A coloured sharp icon: :material-sharp:`settings;3em;sd-text-success`, some more␣
↪text.
- A coloured round icon: :material-round:`settings;3em;sd-text-success`, some more␣
↪text.
- A coloured two-tone icon: :material-twotone:`settings;3em;sd-text-success`, some␣
↪more text.
- A fixed size icon: :material-regular:`data_exploration;24px`, some more text.
```

This directive renders as follows:

- A regular icon: , some more text
- A coloured regular icon: , some more text.
- A coloured outline icon: , some more text.
- A coloured sharp icon: , some more text.
- A coloured round icon: , some more text.
- A coloured two-tone icon: , some more text.
- A fixed size icon: , some more text.

Icon-octicon

```
A coloured icon: :octicon:`report;1em;sd-text-info`, some more text.
```

This directive renders as follows:

A coloured icon: , some more text.

Tab-basic

```
.. tab-set::

    .. tab-item:: Label1

        Content 1

    .. tab-item:: Label2

        Content 2
```

This directive renders as follows:

Label1

Content 1

Label2

Content 2

Tab-code-set

```
.. tab-set-code::

    .. literalinclude:: ./snippet.py
        :language: python

    .. code-block:: javascript

        a = 1;
```

This directive renders as follows:

PYTHON

```
# Copyright (C) 2021 - 2026 Synopsys, Inc. and ANSYS, Inc. All rights reserved.
# SPDX-License-Identifier: Apache-2.0
#
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
```

(continues on next page)

(continued from previous page)

```
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

"""Sample functions for ansys-sphinx-theme."""


def func(arg1, arg2):
    """Summary line <should be one one line>.

    Extended description of function.  Can span multiple lines and
    provides a general overview of the function.

    .. warning::
       Use the ``.. warning::`` directive within the doc-string for any
       warnings you would like to explicitly state.  For example, if
       this method will be deprecated in the next release.

    Parameters
    ----------
    arg1 : int
        Description of arg1.
    arg2 : str
        Description of arg2.

    Returns
    -------
    bool
        Description of return value.

    Examples
    --------
    >>> func(1, "foo")
    True

    """
    return True
```

JAVASCRIPT

```
a = 1;
```

Tab-options

```
.. tab-set::
    :class: class-set
```

(continues on next page)

(continued from previous page)

```
    .. tab-item:: **Label**
        :name: tab-options.txt
        :selected:
        :class-container: class-container
        :class-label: class-label
        :class-content: class-content

        Content

:ref:`tab-options.txt`, :ref:`text <tab-options.txt>`
```

This directive renders as follows:

Label

Content

Label, *text*

Tab-sync

```
.. tab-set::
    :sync-group: category

    .. tab-item:: Label1
        :sync: key1

        Content 1

    .. tab-item:: Label2
        :sync: key2

        Content 2

.. tab-set::
    :sync-group: category

    .. tab-item:: Label1
        :sync: key1

        Content 1

    .. tab-item:: Label2
        :sync: key2

        Content 2
```

This directive renders as follows:

Label1

Content 1

Label2

Content 2

Label1

Content 1

Label2

Content 2

3.2.2 Jupyter notebook - nbsphinx

This example shows how the Ansys Sphinx Theme renders a Jupyter notebook using the `nbspinx` extension.

Download this example as a `Jupyter notebook`.

Nbsphinx example

This example renders a Jupyter notebook using the `nbsphinx` extension.

Plot a simple sphere using PyVista.

[1]:
```
import pyvista as pv

pv.set_jupyter_backend("html")

sphere = pv.Sphere()
sphere.plot()
```

```
<trame_pyvista.jupyter.EmbeddableWidget object at 0x7f731ef9e660>
```

[2]:
```
plotter = pv.Plotter(notebook=True)
plotter.add_mesh(sphere, color="white", show_edges=True)
plotter.title = "3D Sphere Visualization"
plotter.show()
```

```
<trame_pyvista.jupyter.EmbeddableWidget object at 0x7f730d3a4cd0>
```

Figures with Matplotlib

This example shows how to render a figure using Matplotlib.

[3]:
```
import matplotlib.pyplot as plt
import numpy as np

time = np.linspace(0, 2 * np.pi, 100)
```

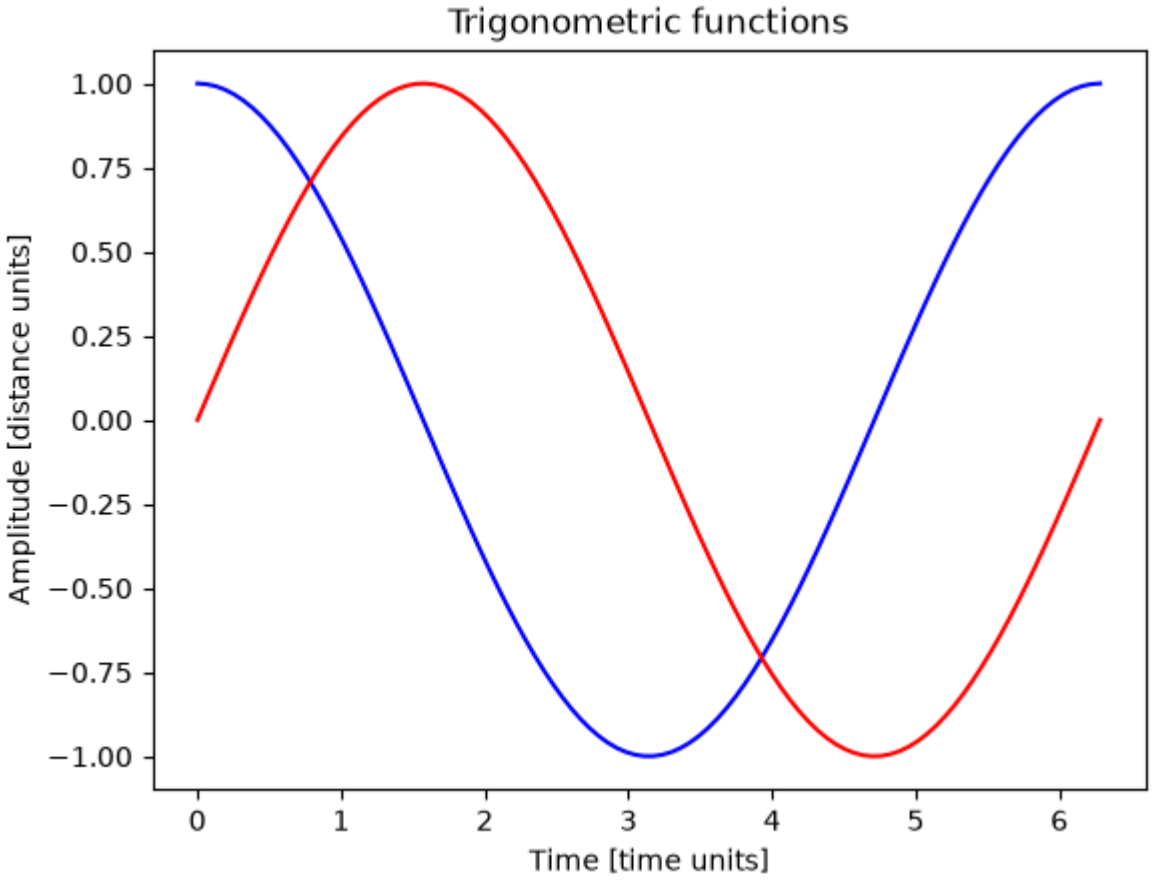
(continues on next page)

(continued from previous page)

```
fig, ax = plt.subplots()
ax.plot(time, np.cos(time), color="blue", label=r"$\cos{(t)}$")
ax.plot(time, np.sin(time), color="red", label=r"$\sin{(t)}$")

ax.set_xlabel("Time [time units]")
ax.set_ylabel("Amplitude [distance units]")
ax.set_title("Trigonometric functions")

plt.show()
```



Figures with Plotly

This example shows how to render a figure using Plotly.

[4]:
```
from IPython.display import HTML, display
import plotly.graph_objs as go

time = np.linspace(0, 2 * np.pi, 100)

cos_trace = go.Scatter(x=time, y=np.cos(time), mode="lines", name="cos(t)")
sin_trace = go.Scatter(x=time, y=np.sin(time), mode="lines", name="sin(t)")

fig = go.Figure(data=[cos_trace, sin_trace])
```

(continues on next page)

(continued from previous page)

```
# Render as embeddable HTML so nbsphinx can always display it in static docs.
display(HTML(fig.to_html(include_plotlyjs="cdn", full_html=False)))
```

```
<IPython.core.display.HTML object>
```

Render equations using the IPython `math` module.

[5]:
```
from IPython.display import Math, display

# LaTeX formatted equation
equation = r"\int\limits_{-\infty}^\infty f(x) \delta(x - x_0) \, dx = f(x_0)"
# Display the equation
display(Math(equation))
```

$$\int_{-\infty}^{\infty} f(x)\delta(x - x_0)\,dx = f(x_0)$$

[6]:
```
from IPython.display import Latex

Latex(r"This is a \LaTeX{} equation: $a^2 + b^2 = c^2$")
```

[6]: This is a LaTeX equation: $a^2 + b^2 = c^2$

Render a table in markdown.

This is an example to render the table inside the notebook

A	B	A and B
False	False	False
True	False	False
False	True	False
True	True	True

Render a data frame

[7]:
```
import pandas as pd

# Create a dictionary of data
data = {
    "A": [True, False, True, False],
    "B": [False, True, False, True],
    "C": [True, True, False, False],
}

# Create DataFrame from the dictionary
df = pd.DataFrame(data)

# Display the DataFrame
df.head()
```

[7]:
```
       A      B      C
0   True  False   True
1  False   True   True
2   True  False  False
3  False   True  False
```

3.2.3 Sphinx-Gallery

This example shows how the Ansys Sphinx Theme renders examples using Sphinx-Gallery.

Sphinx-Gallery

This example shows how to add a new example when using Sphinx-Gallery.

To use Sphinx-Gallery, first install the package with this command:

```
pip install sphinx-gallery
```

Then, add the package to the `extensions` variable in your Sphinx `conf.py` file:

```
extensions = [
    "sphinx_gallery.gen_gallery",
]
```

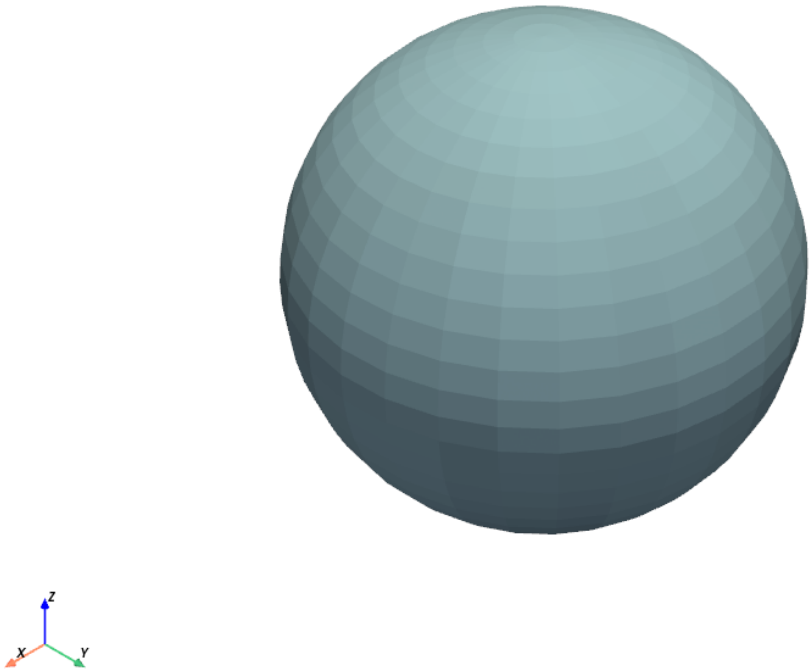
Plot a simple sphere using PyVista

This code plots a simple sphere using PyVista.

```
import pyvista as pv

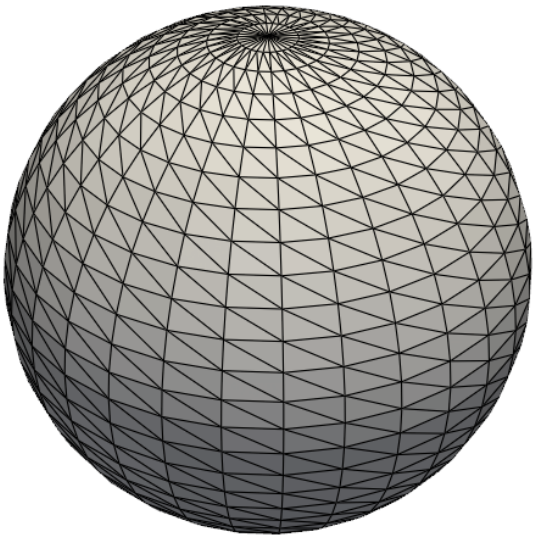
pv.set_jupyter_backend("html")

sphere = pv.Sphere()
sphere.plot()
```



Plot a simple sphere using PyVista with a plotter

```
plotter = pv.Plotter(notebook=True)
plotter.add_mesh(sphere, color="white", show_edges=True)
plotter.title = "3D Sphere Visualization"
plotter.show()
```



```
<trame_pyvista.jupyter.EmbeddableWidget object at 0x7f3760bbb4d0>
```

Figures with Matplotlib

This example shows how to render a figure using Matplotlib.

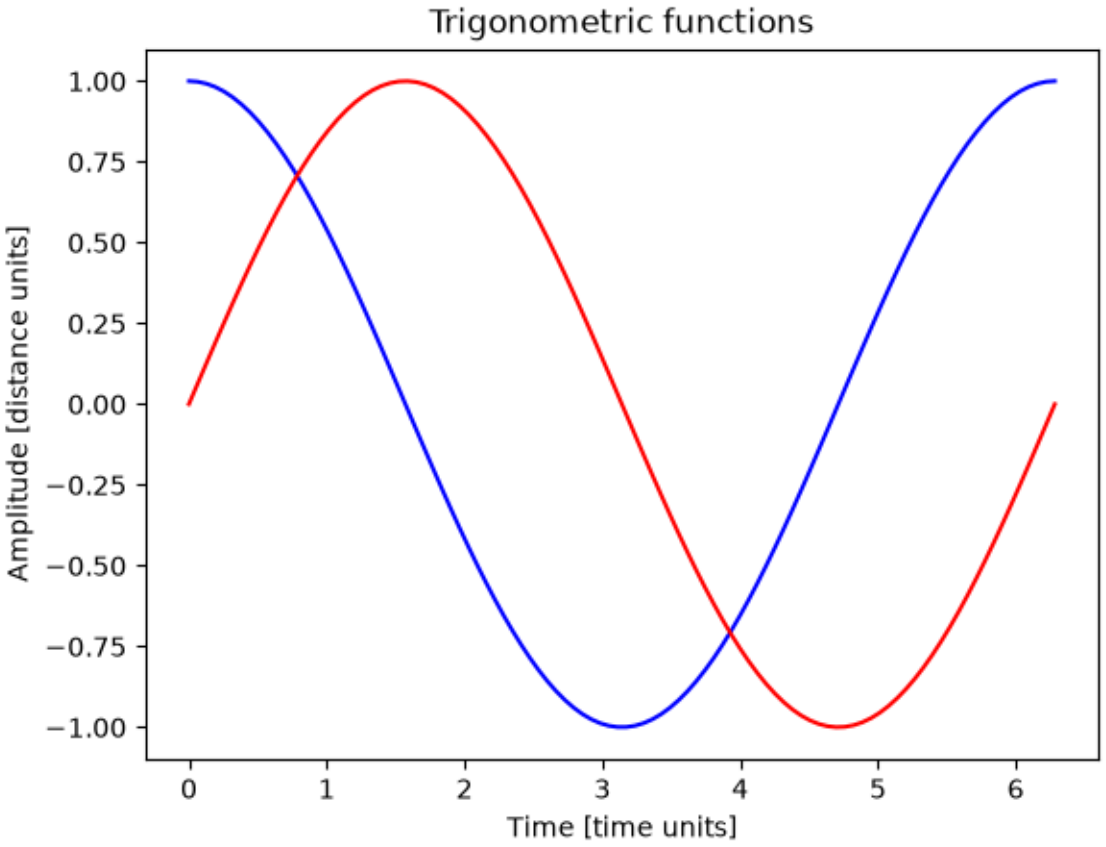
```
import matplotlib.pyplot as plt
import numpy as np

time = np.linspace(0, 2 * np.pi, 100)

fig, ax = plt.subplots()
ax.plot(time, np.cos(time), color="blue", label=r"$\cos{(t)}$")
ax.plot(time, np.sin(time), color="red", label=r"$\sin{(t)}$")

ax.set_xlabel("Time [time units]")
ax.set_ylabel("Amplitude [distance units]")
ax.set_title("Trigonometric functions")

plt.show()
```



Figures with Plotly

This example shows how to render a figure using Plotly.

```
import plotly.graph_objs as go

# More info: https://plotly.com/python/renderers/

time = np.linspace(0, 2 * np.pi, 100)

cos_trace = go.Scatter(x=time, y=np.cos(time), mode="lines", name="cos(t)")
sin_trace = go.Scatter(x=time, y=np.sin(time), mode="lines", name="sin(t)")

fig = go.Figure(data=[cos_trace, sin_trace])

fig
```

```
import numpy as np
import plotly.express as px

df = px.data.tips()
fig = px.bar(
    df,
```

(continues on next page)

(continued from previous page)

```
    x="sex",
    y="total_bill",
    facet_col="day",
    color="smoker",
    barmode="group",
    template="presentation+plotly",
)
fig.update_layout(height=400)
fig
```

Render equations using IPython `math`

This example shows how to render equations using the IPython `math` module.

```
from IPython.display import Math, display

# LaTeX formatted equation
equation = r"\int\limits_{-\infty}^\infty f(x) \delta(x - x_0) \, dx = f(x_0)"
# Display the equation
display(Math(equation))
```

```
<IPython.core.display.Math object>
```

```
from IPython.display import Latex

Latex(r"This is a \LaTeX{} equation: $a^2 + b^2 = c^2$")
```

```
<IPython.core.display.Latex object>
```

Render a table

This is an example to render a table with Sphinx-Gallery.

A	B	A and B
False	False	False
True	False	False
False	True	False
True	True	True

Render a table using pandas

```
import pandas as pd

# Create a dictionary of data
data = {
    "A": [True, False, True, False],
    "B": [False, True, False, True],
    "C": [True, True, False, False],
}
```

(continues on next page)

(continued from previous page)

```
# Create DataFrame from the dictionary
df = pd.DataFrame(data)

# Display the DataFrame
df.head()
```

sphinx_gallery_tags = ["sphinx-gallery-rendered"]

Total running time of the script: (0 minutes 6.048 seconds)

Tags: sphinx-gallery-rendered

3.2.4 Sphinx AutoAPI

This example shows how the Ansys Sphinx Theme renders API documentation using the Sphinx AutoAPI extension.

API reference

This section describes ansys_sphinx_theme endpoints, their capabilities, and how to interact with them programmatically.

The `examples` package

Summary

Submodules

sample_func	Sample functions for ansys-sphinx-theme.
samples	Sample classes and functions for ansys-sphinx-theme.
type_hint_example	Module containing an example function using type hinting.

The `sample_func.py` module

Summary

Functions

func	Summary line <should be one one line>.

examples.sample_func.func

examples.sample_func.**func**(*arg1*, *arg2*)

Summary line <should be one one line>.

Extended description of function. Can span multiple lines and provides a general overview of the function.

> **Warning**
>
> Use the `.. warning::` directive within the doc-string for any warnings you would like to explicitly state. For example, if this method will be deprecated in the next release.

Parameters

arg1
[int] Description of arg1.

arg2
[str] Description of arg2.

Returns

bool
Description of return value.

Examples

```
>>> func(1, "foo")
True
```

Description

Sample functions for ansys-sphinx-theme.

The `samples.py` module

Summary

Classes

ExampleClass	The summary line for a class docstring should fit on one line.
Complex	Custom implementation of a complex number.
ExampleDataClass	An example dataclass.
ExamplePydanticClass	An example Pydantic class.

ExampleClass

class examples.samples.**ExampleClass**(*param1*, *param2*, *param3=0*)

The summary line for a class docstring should fit on one line.

Attributes should be documented inline with the attribute's declaration.

Properties created with the `@property` decorator should be documented in the property's getter method.

Parameters

param1
[str] Description of *param1*.

param2
[list of str] Description of *param2*. Multiple lines are supported.

param3
[int, optional] Description of *param3*.

Examples

An example of how to initialize this class should be given.

```
>>> from ansys_sphinx_theme import samples
>>> example = samples.ExampleClass("mystr", ["apple", "orange"], 3)
```

Overview

Methods

example_method	Class methods are similar to regular functions.

Properties

readonly_property	Properties should be documented in their getter method.
readwrite_property	Set or return the readwrite property.

Attributes

attr1
attr2
attr3
attr4
attr5

Special methods

__special__	By default special members with docstrings are not included.
__special_without_docstring__	

Import detail

```
from examples.samples import ExampleClass
```

Property detail

property ExampleClass.**readonly_property**: **str**

Properties should be documented in their getter method.

Examples

```
>>> example.readonly_property
"readonly_property"
```

property ExampleClass.**readwrite_property**

Set or return the readwrite property.

Properties with both a getter and setter should only be documented in their getter method.

If the setter method contains notable behavior, it should be mentioned here.

Examples

```
>>> example.readwrite_property
"readwrite_property"
```

```
>>> example.readwrite_property = "hello world"
>>> example.readwrite_property
'hello world'
```

Attribute detail

ExampleClass.**attr1**

ExampleClass.**attr2**

ExampleClass.**attr3 = 0**

ExampleClass.**attr4 = ['attr4']**

ExampleClass.**attr5 = None**

Method detail

ExampleClass.**example_method**(*param1*, *param2*)

Class methods are similar to regular functions.

Parameters

param1
[str] The first parameter.

param2
[str] The second parameter.

Returns

bool
`True` if successful, `False` otherwise.

Notes

Do not include the `self` parameter in the `Parameters` section.

Examples

```
>>> example.example_method("foo", "bar")
True
```

ExampleClass.**__special__**()

By default special members with docstrings are not included.

Special members are any methods or attributes that start with and end with a double underscore. Any special member with a docstring will be included in the output, if napoleon_include_special_with_doc is set to True.

This behavior can be enabled by changing the following setting in Sphinx's conf.py:

```
napoleon_include_special_with_doc = True
```

ExampleClass.**__special_without_docstring__**()

Complex

class examples.samples.**Complex**(*real*, *imag=0.0*)

Bases: object

Custom implementation of a complex number.

Parameters

real
[float] Real component of the complex number.

imag
[float, optional] Imaginary component of the complex number.

Examples

```
>>> my_num = Complex(real=1, imag=-1.0)
>>> my_num
(1.0 + 1.0j)
```

Overview

Properties

real	Real component of this complex number.
imag	Real component of this complex number.
abs	Return the absolute value of this number.

Special methods

__add__	Add two complex numbers.
__sub__	Subtract two complex numbers.
__mul__	Multiply two complex numbers.
__truediv__	Divide two complex numbers.
__repr__	

Import detail

```
from examples.samples import Complex
```

Property detail

property Complex.**real**

Real component of this complex number.

Examples

```
>>> my_num = Complex(real=1, imag=-1.0)
>>> my_num.real
1.0
```

property Complex.**imag**

Real component of this complex number.

Examples

```
>>> my_num = Complex(real=1, imag=-1.0)
>>> my_num.imag
-1.0
```

Set the imaginary component

```
>>> my_num.imag = 2.0
>>> my_num.imag
2.0
```

property Complex.**abs**

Return the absolute value of this number.

Examples

```
>>> my_num = Complex(real=1, imag=1.0)
>>> my_num.abs
```

Method detail

Complex.**__add__**(*other*)

Add two complex numbers.

Complex.**__sub__**(*other*)

Subtract two complex numbers.

Complex.**__mul__**(*other*)

Multiply two complex numbers.

Complex.**__truediv__**(*other*)

Divide two complex numbers.

Complex.**__repr__**()

ExampleDataClass

class examples.samples.**ExampleDataClass**

An example dataclass.

Parameters

value
[float] A mandatory parameter.

value_with_default
[float, optional] An optional parameter with a default value of 3.14.

value_with_default_field
[float, optional] An optional parameter using field for the default value. Default is 3.14.

Examples

```
>>> example = ExampleDataClass(value=42)
>>> example.value
42
>>> example.value_with_default
3.14
>>> example.value_with_default_field
3.14
```

Overview

Attributes

value	A mandatory parameter.
value_with_default	An optional parameter with a default value.
value_with_default_field	An optional parameter using field for the default value.

Import detail

```
from examples.samples import ExampleDataClass
```

Attribute detail

ExampleDataClass.**value: float**

A mandatory parameter.

ExampleDataClass.**value_with_default: float = 3.14**

An optional parameter with a default value.

ExampleDataClass.**value_with_default_field: float = 3.14**

An optional parameter using field for the default value.

ExamplePydanticClass

class examples.samples.**ExamplePydanticClass**(/, ***data: Any*)

Bases: pydantic.BaseModel

An example Pydantic class.

Parameters

value
[float] A required parameter.

value_with_default1
[float, optional] A simple optional parameter with a default value of 3.14.

value_with_default2
[float, optional] A simple optional parameter using Field for the default value and description. Default is 3.14.

value_with_constraints
[float, optional] An optional parameter with constraints using Field for the default value, description and constraints. Default is 10.0, with constraints: 0.0 <= value <= 100.0.

Examples

```
>>> example = ExamplePydanticClass(value=42)
>>> example.value
42
>>> example.value_with_default1
3.14
>>> example.value_with_default2
3.14
>>> example.value_with_constraints
10.0
```

Overview

Attributes

value	A required parameter.
value_with_default1	A simple optional parameter with a default value.
value_with_default2	A simple optional parameter using field for default value and description.
value_with_constraint	A optional parameter with constraints using field for default value, description and constraints.

Import detail

```
from examples.samples import ExamplePydanticClass
```

Attribute detail

ExamplePydanticClass.**value**: float

A required parameter.

ExamplePydanticClass.**value_with_default1: float = 3.14**

A simple optional parameter with a default value.

ExamplePydanticClass.**value_with_default2: float**

A simple optional parameter using field for default value and description.

ExamplePydanticClass.**value_with_constraints: float**

A optional parameter with constraints using field for default value, description and constraints.

Description

Sample classes and functions for ansys-sphinx-theme.

The `type_hint_example.py` module

Summary

Functions

type_hint_func	Summary containing the function description.

examples.type_hint_example.type_hint_func

examples.type_hint_example.**type_hint_func**(*param1: int = 1*, *param2: str = 'test'*, *param3: int | float = 1*) → bool

Summary containing the function description.

Extended description of the function. Can span multiple lines and provides a general overview of the function.

Parameters

param1
Description of an integer parameter.

param2
Description of a string parameter.

param3
Parameter that can be either int or float using Union (typing).

Returns

bool
Description of the returned value.

Examples

```
>>> func(1, "foo", 1)
True
```

Description

Module containing an example function using type hinting.

Description

A sub-package containing various examples for checking their rendering.

3.2.5 Table

The table directive with ansys sphinx theme allows for rendering of tables. There are different types of tables, such as the data table, longtable-centered, and table-centered, each serving different purposes.

Normal table

Table 1: Truth table for “not”

A	B	C
False	True	False
	False	True
False	True	False
	False	True
False	True	False
	True	False
	False	True

Data table

This is an example of a data table that can be rendered using the table directive. It consists of three columns representing the variables A, B, and A and B respectively. Each row represents a different combination of True and False for variables A and B. The *datatable* class can be used to style the data table.

```
.. table::
    :class: datatable

    =====  =====  =======
    A      B      A and B
    =====  =====  =======
    False  False  False
    True   False  False
    False  True   False
    True   True   True
    =====  =====  =======
```

A	B	A and B
False	False	False
True	False	False
False	True	False
True	True	True

Longtable-centered

The longtable-centered class can be used to create a table that spans multiple pages and is centered horizontally. This is useful for tables that have a large number of rows or columns. Here is an example of a longtable-centered:

```
.. table::
    :class: longtable-centered

    +---------------------------+--------------------+
    |                           | MAPDL Command      |
    +===========================+====================+
    | **GUI commands**          | * ``*ASK``         |
    |                           |                    |
    | **GUI commands**          | * ``*ASK``         |
    |                           |                    |
    | **GUI commands**          | * ``*ASK``         |
    +---------------------------+--------------------+
```

	MAPDL Command
GUI commands	• *ASK
GUI commands	• *ASK
GUI commands	• *ASK

Table-centered

The table-centered class can be used to create a table that is horizontally centered. This is useful for tables that have only a few columns. Here is an example of a table-centered:

```
.. table::
    :class: table-centered

    +---------------------------+--------------------+
    |                           | MAPDL Command      |
    +===========================+====================+
    | **GUI commands**          | * ``*ASK``         |
    +---------------------------+--------------------+
    | **GUI commands**          | * ``*ASK``         |
    +---------------------------+--------------------+
    | **GUI commands**          | * ``*ASK``         |
    +---------------------------+--------------------+
```

	MAPDL Command
GUI commands	• *ASK
GUI commands	• *ASK
GUI commands	• *ASK

3.2.6 Admonitions

Admonitions are specially formatted blocks that highlight important information in the documentation. This page shows how the Ansys Sphinx Theme renders admonitions.

The examples shown below are derived from the PyData Sphinx Theme project, copyrighted by pandas, 2018. Original examples can be found in Examples in the PyData Sphinx Theme documentation. These examples are licensed under the BSD 3-Clause License.

The examples included below in this page were originally copyrighted and licensed as follows:

- Copyright (c) 2021, Pradyun Gedam. Licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (SPDX-License-Identifier: CC-BY-SA-4.0), as specified by the PyData Sphinx Theme in their examples.

Admonitions

Sphinx provides several different types of admonitions.

`topic`

This is a topic.

This is what admonitions are a special case of, according to the docutils documentation.

We also support *italic*, **bold**, `code`, links, and more.

`admonition`

The one with the custom titles

It's got a certain charm to it.

We also support *italic*, **bold**, `code`, links, and more.

`attention`

Attention

Climate change is real.

We also support *italic*, **bold**, `code`, links, and more.

`caution`

Caution

Cliff ahead: Don't drive off it.

We also support *italic*, **bold**, `code`, links, and more.

`danger`

Danger

Mad scientist at work!

We also support *italic*, **bold**, `code`, links, and more.

`error`

Error

Does not compute.

We also support *italic*, **bold**, `code`, links, and more.

`hint`

Hint

Insulators insulate, until they are subject to _____ voltage.

We also support *italic*, **bold**, `code`, links, and more.

`important`

Important

Tech is not neutral, nor is it apolitical.

We also support *italic*, **bold**, `code`, links, and more.

`note`

Note

This is a note.

We also support *italic*, **bold**, `code`, links, and more.

`seealso`

See also

Other relevant information.

We also support *italic*, **bold**, `code`, links, and more.

`tip`

Tip

25% if the service is good.

We also support *italic*, **bold**, `code`, links, and more.

todo

Todo

This needs the `sphinx.ext.todo` extension.

We also support *italic*, **bold**, `code`, links, and more.

warning

Warning

Reader discretion is strongly advised.

We also support *italic*, **bold**, `code`, links, and more.

versionadded

Added in version v0.1.1: Here's a version added message.

We also support *italic*, **bold**, `code`, links, and more.

versionchanged

Changed in version v0.1.1: Here's a version changed message.

We also support *italic*, **bold**, `code`, links, and more.

deprecated

Deprecated since version v0.1.1: Here's a deprecation message.

We also support *italic*, **bold**, `code`, links, and more.

versionremoved

Removed in version v0.1.1: Here's a version removed message.

We also support *italic*, **bold**, `code`, links, and more.

CHAPTER

FOUR

RELEASE NOTES

4.1 1.12.1 (October 26, 2026)

Miscellaneous

Docs: add documentation for news and resources option	#1192

4.2 1.12.0 (October 22, 2026)

Added

Feat: news and resources architecture	#1041
Fix: show actual section title in sidebar	#1101
Chore(version): bump version 1.12.dev0	#1116
Feat: migrate browser tests from JavaScript playwright to Python playwright	#1165

Dependencies

Build: update pydata-sphinx-theme requirement from <0.21,>=0.15.4 to >=0.15.4,<0.22	#1112
Build: bump pydantic from 2.13.4 to 2.13.5	#1114
Build: bump ty from 0.0.75 to 0.0.77	#1119
Build: bump tox from 4.60.1 to 4.61.1	#1120
Build: bump tox from 4.61.1 to 4.61.2	#1125
Build: bump ty from 0.0.77 to 0.0.78	#1126
Build: bump tox from 4.61.2 to 4.61.3	#1128
Build: bump ty from 0.0.78 to 0.0.79	#1129
Build: bump ty from 0.0.79 to 0.0.80	#1131
Build: bump tox from 4.61.3 to 4.61.4	#1132
Build: bump pyvista from 0.48.4 to 0.49.0	#1133
Build: bump ty from 0.0.80 to 0.0.81	#1135
Build: bump ty from 0.0.81 to 0.0.82	#1138
Build: bump tox from 4.61.4 to 4.61.5	#1139
Build: bump numpydoc from 1.10.0 to 1.11.0	#1140
Build: bump pandas from 3.0.5 to 3.0.6	#1141
Build: bump sphinx-gallery from 0.21.0 to 0.22.0	#1142
Build: bump sphinx-gallery from 0.22.0 to 0.22.1	#1143

continues on next page

Table 1 – continued from previous page

Build: bump plotly from 6.9.0 to 7.0.0	#1144
Build: bump tox from 4.61.5 to 4.63.0	#1145
Build: bump notebook from 7.6.2 to 7.6.3	#1146
Build: bump ty from 0.0.82 to 0.0.83	#1147
Build: bump plotly from 7.0.0 to 7.1.0	#1148
Build: bump ty from 0.0.83 to 0.0.84	#1152
Build: bump tox from 4.63.0 to 4.64.0	#1153
Build: bump tox from 4.64.0 to 4.64.3	#1154
Build: update pydata-sphinx-theme requirement from <0.22,>=0.15.4 to >=0.15.4,<0.23	#1155
Build: bump tox from 4.64.3 to 4.64.5	#1156
Build: bump tox from 4.64.5 to 4.64.7	#1158
Build: bump tox from 4.64.7 to 4.64.8	#1160
Build: bump jupytext from 1.19.5 to 1.19.6	#1161
Build: bump ty from 0.0.84 to 0.0.85	#1162
Build: bump tox from 4.64.8 to 4.64.9	#1163
Build: bump quarto-cli from 1.10.18 to 1.10.19	#1167
Build: bump tox from 4.64.9 to 4.64.10	#1168
Build: bump ty from 0.0.85 to 0.0.86	#1171
Build: bump pydantic from 2.13.5 to 2.14.0	#1172
Build: update pydata-sphinx-theme requirement from <0.23,>=0.15.4 to >=0.15.4,<0.24	#1173
Build: bump pyvista from 0.49.0 to 0.49.1	#1175
Build: bump pydantic from 2.14.0 to 2.14.1	#1176
Build: bump tox from 4.64.10 to 4.65.3	#1177
Build: bump ty from 0.0.86 to 0.0.87	#1178
Build: bump tox from 4.65.3 to 4.65.5	#1180
Build: bump tox from 4.65.5 to 4.65.6	#1181
Build: bump ty from 0.0.87 to 0.0.88	#1182
Build: bump ty from 0.0.88 to 0.0.89	#1185

Maintenance

Ci: bump the actions group with 14 updates	#1115, #1127
Chore: pre-commit automatic update	#1124, #1130, #1137, #1151, #1157, #1164, #1179
Ci: bump astral-sh/setup-uv from 10.0.1 to 10.1.0 in the actions group	#1134
Ci: bump astral-sh/setup-uv from 10.1.0 to 10.2.0 in the actions group	#1149
Ci: bump the actions group across 1 directory with 4 updates	#1174

Miscellaneous

Chore: update CHANGELOG for v1.11.0	#1113
Fix: dark theme search sidebar	#1118
Docs: add pull request template	#1121
Docs: add guidelines for js, css and html	#1122
Docs: fix `notebook` examples plot	#1136
Fix: improve readability by lowering opacity in code blocks	#1150
Maint: remove unwanted `doc-build` dependency	#1170
Fix: add an environment variable for the port in tests	#1184

4.3 1.11.0 (September 02, 2026)

Added

Feat: add weighted score	#928
Migrate to `ty-pre-commit` hook	#1082
Feat: adding MCP Server banner feature	#1105

Dependencies

Bump ty from 0.0.65 to 0.0.67	#1086
Bump notebook from 7.6.1 to 7.6.2	#1089
Build: bump ty from 0.0.67 to 0.0.70	#1090
Build: bump tox from 4.58.0 to 4.59.0	#1092
Build: bump ty from 0.0.70 to 0.0.71	#1094
Build: bump ty from 0.0.71 to 0.0.72	#1096
Build: bump tox from 4.59.0 to 4.60.0	#1097
Build: bump ty from 0.0.72 to 0.0.73	#1100
Build: bump pygithub from 2.9.1 to 2.10.0	#1102
Build: bump sphinx-autoapi from 3.8.0 to 3.8.1	#1106
Build: bump tox from 4.60.0 to 4.60.1	#1107
Build: bump ty from 0.0.73 to 0.0.75	#1109

Maintenance

Update CHANGELOG for v1.10.0	#1078
Bump pypa/gh-action-pypi-publish from 1.14.1 to 1.14.2 in the actions group	#1080
Bump version 1.11.dev0	#1081
Update missing or outdated files	#1083
Pre-commit automatic update	#1085
Ci: bump the actions group across 1 directory with 14 updates	#1087
Chore: pre-commit automatic update	#1091, #1099, #1108
Ci: bump the actions group with 14 updates	#1093, #1104, #1110
Ci: bump the actions group with 15 updates	#1095
Ci: bump astral-sh/setup-uv from 10.0.0 to 10.0.1 in the actions group	#1098

4.4 1.10.0 (August 04, 2026)

Added

Drop python `3.10` and `3.11`	#1021
Announcements with theme	#1023

Fixed

Broken links	#1027

Dependencies

Update pydata-sphinx-theme requirement from <0.19,>=0.15.4 to >=0.15.4,<0.20	#1024
Bump ty from 0.0.49 to 0.0.50	#1025
Bump pandas from 2.3.3 to 3.0.3	#1028
Bump ty from 0.0.50 to 0.0.51	#1030
Bump notebook from 7.5.7 to 7.6.0	#1031
Bump ty from 0.0.51 to 0.0.52	#1032
Bump jupytext from 1.19.3 to 1.19.4	#1033
Bump ty from 0.0.52 to 0.0.53	#1034
Bump tox from 4.55.1 to 4.56.0	#1036
Bump ty from 0.0.53 to 0.0.54	#1037
Bump tox from 4.56.0 to 4.56.1	#1039
Bump ty from 0.0.54 to 0.0.55	#1040
Bump ty from 0.0.55 to 0.0.56	#1042
Bump tox from 4.56.1 to 4.56.2	#1046
Bump ty from 0.0.56 to 0.0.57	#1048
Bump tox from 4.56.2 to 4.56.4	#1049
Bump ty from 0.0.57 to 0.0.58	#1052
Bump plotly from 6.8.0 to 6.9.0	#1053
Update pydata-sphinx-theme requirement from <0.20,>=0.15.4 to >=0.15.4,<0.21	#1054
Bump ty from 0.0.58 to 0.0.59	#1056
Bump ty from 0.0.59 to 0.0.60	#1058
Bump ty from 0.0.60 to 0.0.61	#1060
Bump tox from 4.56.4 to 4.57.0	#1064
Bump jupytext from 1.19.4 to 1.19.5	#1065
Bump notebook from 7.6.0 to 7.6.1	#1067
Bump pandas from 3.0.3 to 3.0.5	#1068
Bump tox from 4.57.0 to 4.57.2	#1070
Bump ty from 0.0.61 to 0.0.63	#1072
Bump tox from 4.57.2 to 4.58.0	#1073
Bump ty from 0.0.63 to 0.0.64	#1074
Bump ty from 0.0.64 to 0.0.65	#1076

Maintenance

Update CHANGELOG for v1.9.0	#1019
Bump version to 1.10.dev0	#1020
Update missing or outdated files	#1022
Bump actions/checkout from 6.0.3 to 7.0.0 in the actions group	#1029
Bump the actions group with 14 updates	#1035, #1055
Bump the actions group with 16 updates	#1047
Bump astral-sh/setup-uv from 8.3.0 to 8.3.1 in the actions group	#1050
Bump astral-sh/setup-uv from 8.3.1 to 8.3.2 in the actions group	#1051
Bump actions/setup-node from 6.4.0 to 7.0.0 in the actions group	#1057
Bump actions/labeler from 6.2.0 to 7.0.0 in the actions group	#1059
Bump actions/checkout from 7.0.0 to 7.0.1 in the actions group	#1061
Bump pypa/gh-action-pypi-publish from 1.14.0 to 1.14.1 in the actions group	#1066
Bump astral-sh/setup-uv from 8.3.2 to 9.0.0 in the actions group	#1071
Bump the actions group with 13 updates	#1077

4.5 1.9.0 (June 18, 2026)

Added

Add default `copyright` options with theme	#1004

Fixed

Copyright update in all files	#1016
Typo in copyright	#1018

Dependencies

Bump requests from 2.33.1 to 2.34.0	#978
Bump requests from 2.34.0 to 2.34.2	#979
Bump tox from 4.53.1 to 4.54.0	#980
Bump ty from 0.0.35 to 0.0.36	#982
Bump ty from 0.0.36 to 0.0.37	#988
Bump jupytext from 1.19.2 to 1.19.3	#991
Bump pyvista from 0.48.2 to 0.48.4	#992
Bump ty from 0.0.37 to 0.0.38	#993
Bump ty from 0.0.38 to 0.0.39	#996
Update pydata-sphinx-theme requirement from <0.18,>=0.15.4 to >=0.15.4,<0.19	#997
Bump ty from 0.0.39 to 0.0.40	#1000
Bump tox from 4.54.0 to 4.55.0	#1001
Bump ty from 0.0.40 to 0.0.42	#1002
Bump tox from 4.55.0 to 4.55.1	#1005
Bump ty from 0.0.42 to 0.0.43	#1007
Bump notebook from 7.5.6 to 7.5.7	#1008
Bump plotly from 6.7.0 to 6.8.0	#1010
Bump ty from 0.0.43 to 0.0.44	#1011
Bump ty from 0.0.44 to 0.0.46	#1013
Bump ty from 0.0.46 to 0.0.48	#1014
Bump ty from 0.0.48 to 0.0.49	#1015

Maintenance

Update CHANGELOG for v1.8.0	#976
Bump version 1.9.dev0	#977
Bump ansys/actions from 10.3.0 to 10.3.1 in the actions group	#981
Update CHANGELOG for v1.8.1	#984
Update CHANGELOG for v1.8.2	#994
Update license to Apache-2.0	#995
Bump ansys/actions from 10.3.1 to 10.3.2 in the actions group	#999
Update license to `Apache-2.0` in `pyproject`	#1003
Bump actions/checkout from 6.0.2 to 6.0.3 in the actions group	#1006
Bump astral-sh/setup-uv from 8.1.0 to 8.2.0 in the actions group	#1009

Test

Add theme dropdown test	#1012

4.6 1.8.2 (May 26, 2026)

Fixed

Collapse the search results bar after results	#986
Target color style	#989

4.7 1.8.1 (May 21, 2026)

Fixed

Cheatsheet configurations in sidebar	#983

4.8 1.8.0 (May 19, 2026)

Added

Use python `3.14`	#938
Add option to globally hide the secondary sidebar page TOC	#943

Fixed

Resolve type checking errors	#856
``backstop`` test config for tablet and pdf build	#882
Search bar for copy-paste	#927
`backstop` tests issue with collapsible sidebar	#955
Sphinx tags style with `ansys-sphinx-theme`	#965
`cheatsheet` extension version	#975

Documentation

Update ``CONTRIBUTORS.md`` with the latest contributors	#883

Dependencies

Bump jupytext from 1.19.0 to 1.19.1	#879
Bump notebook from 7.5.2 to 7.5.3	#880
Bump pyvista[jupyter] from 0.46.5 to 0.47.0	#885
Bump sphinx-autoapi from 3.6.1 to 3.7.0	#888
Bump tox from 4.34.1 to 4.36.0	#890
Bump tox from 4.36.1 to 4.38.0	#892
Bump tox from 4.38.0 to 4.39.0	#897
Bump notebook from 7.5.3 to 7.5.4	#900
Bump pyvista[jupyter] from 0.47.0 to 0.47.1	#901
Bump tox from 4.39.0 to 4.44.0	#902
Bump tox from 4.44.0 to 4.45.0	#905
Bump tox from 4.45.0 to 4.46.0	#907
Bump tox from 4.46.0 to 4.46.3	#908
Bump tox from 4.46.3 to 4.47.0	#911
Bump plotly from 6.5.2 to 6.6.0	#913
Bump tox from 4.47.0 to 4.47.3	#914
Bump tox from 4.47.3 to 4.48.0	#916
Bump sphinx-autoapi from 3.7.0 to 3.8.0	#917

continues on next page

Table 3 – continued from previous page

Bump tox from 4.48.0 to 4.49.0	#918
Bump notebook from 7.5.4 to 7.5.5	#919
Bump tox from 4.49.0 to 4.49.1	#920
Bump tox from 4.49.1 to 4.50.2	#926
Bump pygithub from 2.8.1 to 2.9.0	#929
Bump tox from 4.50.2 to 4.50.3	#930
Bump requests from 2.32.5 to 2.33.0	#931
Bump requests from 2.33.0 to 2.33.1	#933
Bump tox from 4.50.3 to 4.52.0	#936
Update pydata-sphinx-theme requirement from <0.17,>=0.15.4 to >=0.15.4,<0.18	#941
Bump tox from 4.52.0 to 4.52.1	#944
Bump pyvista from 0.47.1 to 0.47.3	#945
Bump plotly from 6.6.0 to 6.7.0	#946
Bump pygithub from 2.9.0 to 2.9.1	#949
Bump pydantic from 2.12.5 to 2.13.0	#950
Bump tox from 4.52.1 to 4.53.0	#951
Bump pydantic from 2.13.0 to 2.13.1	#952
Bump pydantic from 2.13.1 to 2.13.2	#953
Bump pydantic from 2.13.2 to 2.13.3	#956
Bump ty from 0.0.29 to 0.0.32	#958
Bump notebook from 7.5.5 to 7.5.6	#960
Bump sphinx-gallery from 0.20.0 to 0.21.0	#961
Bump ty from 0.0.32 to 0.0.33	#962
Bump ty from 0.0.33 to 0.0.34	#966
Bump tox from 4.53.0 to 4.53.1	#967
Bump pyvista from 0.47.3 to 0.48.0	#968
Bump pydantic from 2.13.3 to 2.13.4	#969
Bump pyvista from 0.48.0 to 0.48.1	#971
Bump pyvista from 0.48.1 to 0.48.2	#972
Bump jupytext from 1.19.1 to 1.19.2	#973
Bump ty from 0.0.34 to 0.0.35	#974

Maintenance

Update CHANGELOG for v1.7.0	#877
Bump version 1.8.dev0	#878
Extend the cool down period up to 5 days for the patch releases.	#881
Bump ansys/actions from 10.2.3 to 10.2.4 in the actions group	#884
Update CHANGELOG for v1.7.1	#896
Update CHANGELOG for v1.7.2	#904
Bump actions/download-artifact from 7.0.0 to 8.0.0 in the actions group	#906
Bump actions/upload-artifact from 6.0.0 to 7.0.0 in the actions group	#909
Bump the actions group with 2 updates	#912
Bump actions/setup-node from 6.2.0 to 6.3.0 in the actions group	#915
Bump actions/download-artifact from 8.0.0 to 8.0.1 in the actions group	#921
Bump astral-sh/setup-uv from 7.3.1 to 7.6.0 in the actions group	#924
Bump the actions group across 1 directory with 3 updates	#939
Bump actions/upload-artifact from 7.0.0 to 7.0.1 in the actions group	#947
Update license metadata in pyproject.toml	#948
Bump astral-sh/setup-uv from 8.0.0 to 8.1.0 in the actions group	#954, #959
Bump actions/setup-node from 6.3.0 to 6.4.0 in the actions group	#957
Bump ansys/actions from 10.2.12 to 10.3.0 in the actions group across 1 directory	#964
Bump actions/labeler from 6.0.1 to 6.1.0 in the actions group	#970

4.9 1.7.2 (March 02, 2026)

Fixed

Cheatsheet logging and remove commands	#903

4.10 1.7.1 (February 25, 2026)

Fixed

Add flexible dependency versions for ``Python 3.10+`` support and new "all" extras target	#893

Dependencies

Bump tox from 4.36.0 to 4.36.1	#891

Maintenance

Bump the actions group across 1 directory with 2 updates	#887

4.11 1.7.0 (January 28, 2026)

Added

Cleanup ``js``	#798
Add backstop js test	#804
Use dependency groups	#823
Add dataclass and pydantic class example	#832

Fixed

Style fixes with navigation dropdown	#785
Update contact mail	#821
Doc-deploy-pr	#825
Update license year	#862
Sphinx design version	#872
Workflow layout	#874

Documentation

Update ``CONTRIBUTORS.md`` with the latest contributors	#835
Update CONTRIBUTING.md	#846

Dependencies

Bump pyvista[jupyter] from 0.46.2 to 0.46.3	#782
Bump jupytext from 1.17.2 to 1.17.3	#786
Bump tox from 4.28.4 to 4.29.0	#787
Bump pygithub from 2.7.0 to 2.8.1	#789
Bump tox from 4.29.0 to 4.30.2	#791
Bump pyyaml from 6.0.2 to 6.0.3	#800
Bump notebook from 7.4.5 to 7.4.7	#802
Bump pandas from 2.3.2 to 2.3.3	#803
Bump tox from 4.30.2 to 4.30.3	#806
Bump plotly from 6.3.0 to 6.3.1	#807
Bump tox from 4.30.3 to 4.31.0	#819
Bump jupytext from 1.17.3 to 1.18.1	#822
Update to tox==4.32.0	#827
Bump pyvista[jupyter] from 0.46.3 to 0.46.4	#828
Bump plotly from 6.3.1 to 6.4.0	#831
Bump pydantic from 2.12.3 to 2.12.4	#834
Bump plotly from 6.4.0 to 6.5.0	#836
Bump notebook from 7.4.7 to 7.5.0	#837
Bump pydantic from 2.12.4 to 2.12.5	#841
Bump nbsphinx from 0.9.7 to 0.9.8	#842
Bump numpydoc from 1.9.0 to 1.10.0	#844
Bump sphinx-gallery from 0.19.0 to 0.20.0	#845
Bump notebook from 7.5.0 to 7.5.1	#857

continues on next page

Table 4 – continued from previous page

Bump sphinx-theme-builder[cli] from 0.2.0b2 to 0.2.0	#858
Bump sphinx-theme-builder[cli] from 0.2.0 to 0.3.2	#859
Bump tox from 4.32.0 to 4.33.0	#860
Bump plotly from 6.5.0 to 6.5.1	#861
Bump notebook from 7.5.1 to 7.5.2	#864
Bump plotly from 6.5.1 to 6.5.2	#865
Bump pyvista[jupyter] from 0.46.4 to 0.46.5	#866
Bump tox from 4.33.0 to 4.34.1	#868
Bump jupytext from 1.18.1 to 1.19.0	#870

Maintenance

Update CHANGELOG for v1.6.0	#781
Bump version 1.7.dev0	#783
Bump the actions group with 3 updates	#793, #871
Update CHANGELOG for v1.6.1	#797
Bump ansys/actions from 10.0.20 to 10.1.1 in the actions group	#799
Bump ansys/actions from 10.1.1 to 10.1.2 in the actions group	#801
Bump ansys/actions from 10.1.2 to 10.1.4 in the actions group	#808
Add pull request preview action	#813
Update CHANGELOG for v1.6.2	#817
Update CHANGELOG for v1.6.3	#818
Add `zizmor` security actions	#820
Bump actions/download-artifact from 5.0.0 to 6.0.0 in the actions group	#826
Bump ansys/actions from 10.1.4 to 10.1.5 in the actions group	#829
Bump actions/checkout from 5.0.0 to 6.0.0 in the actions group	#839
Update missing or outdated files	#843
Update doc-deploy-pr configuration in workflow	#847
Update CHANGELOG for v1.6.4	#850
Bump the actions group across 1 directory with 3 updates	#851
Update workflow	#852
Update workflow dependencies	#853
Update job dependencies	#854
Bump actions/checkout from 6.0.1 to 6.0.2 in the actions group	#869
Fix labels	#876

4.12 1.6.4 (December 09, 2025)

Fixed

Index anchors	#848

4.13 1.6.3 (October 08, 2025)

Fixed

Logo name	#816

4.14 1.6.2 (October 08, 2025)

Added

Adapt new logo	#811

Fixed

Add flexible range for the ``pyyaml`` version	#814

Dependencies

Bump sphinx-autoapi from 3.6.0 to 3.6.1	#810

4.15 1.6.1 (September 11, 2025)

Fixed

Home link reference	#795

4.16 1.6.0 (August 27, 2025)

Added

Dropdown for navigation bar	#723
Add physics meta tag to headers	#751
Update actions	#756
Add multiple tags for *meta-data*	#758

Fixed

the search title case	#715
Cache of autoapi options with ansys sphinx theme	#728
Raise exception for deprecated index pattern	#729
Remove the deprecated pymeilisearch options	#733
Update the literal *codespace* to *monospace* font	#744
Change the background color	#746
Add padding on top for breadcrumbs	#752
Minor improvements in `jinja` auto escape and `subprocess` call	#754
Remove deprecated theme option from code base	#755
Add home section	#769
Search bar display at `navbar_end`	#780

Documentation

Update `CONTRIBUTORS.md` with the latest contributors	#718, #767

Dependencies

bump jupytext from 1.17.1 to 1.17.2	#719
Bump pandas from 2.2.3 to 2.3.0	#724
Bump requests from 2.32.3 to 2.32.4	#727
Bump tox from 4.26.0 to 4.27.0	#732
Bump numpydoc from 1.8.0 to 1.9.0	#735
Bump plotly from 6.1.2 to 6.2.0	#738
Bump notebook from 7.4.3 to 7.4.4	#743
Bump pandas from 2.3.0 to 2.3.1	#750
Bump pyvista[jupyter] from 0.45.2 to 0.45.3	#753
Bump tox from 4.27.0 to 4.28.0	#757
Bump tox from 4.28.0 to 4.28.1	#759
Bump tox from 4.28.1 to 4.28.3	#760
Bump pygithub from 2.6.1 to 2.7.0	#762
Bump tox from 4.28.3 to 4.28.4	#763
Bump notebook from 7.4.4 to 7.4.5	#770
Bump pyvista[jupyter] from 0.45.3 to 0.46.0	#771
Bump pyvista[jupyter] from 0.46.0 to 0.46.1	#773
Bump plotly from 6.2.0 to 6.3.0	#774
Bump requests from 2.32.4 to 2.32.5	#777
Bump pandas from 2.3.1 to 2.3.2	#778
Bump pyvista[jupyter] from 0.46.1 to 0.46.2	#779

Maintenance

update CHANGELOG for v1.5.0	#711
bump version 1.6.dev0	#712
update the actions to commit hash	#717
update CHANGELOG for v1.5.1	#720
update CHANGELOG for v1.5.2	#722
Bump ansys/actions from 9.0.13 to 10.0.8 in the actions group	#726
Bump ansys/actions from 10.0.8 to 10.0.11 in the actions group	#731
Bump ansys/actions from 10.0.11 to 10.0.12 in the actions group	#742
Update changelog for v1.5.3	#748
Bump ansys/actions from 10.0.12 to 10.0.13 in the actions group	#761
Add *trusted publishers pypi release*	#766
Bump the actions group with 2 updates	#772
Bump actions/checkout from 4.2.2 to 5.0.0 in the actions group	#776

4.17 1.5.3 (July 02, 2025)

Fixed

Add the search button to `navbar end` by default	#747

4.18 1.5.2 (June 02, 2025)

Fixed

remove font weight on highlight	#721

4.19 1.5.1 (June 02, 2025)

Fixed

revert the main content style	#714

4.20 1.5.0 (May 28, 2025)

Added

add multi index searching and filtering	#674
small layout adjustments	#691
add customization instructions for SCSS files in developer documentation	#694
add 'serve' command to Makefile and make.bat for serving documentation	#698

Fixed

smaller screen view and search for mobile views	#689
style for back to top button	#693
url for fetch extra source file	#707

Documentation

update the multi-index docs	#708

Dependencies

bump notebook from 7.4.1 to 7.4.2	#684
update snowballstemmer requirement from <3 to <4	#688
bump pyvista[jupyter] from 0.45.0 to 0.45.1	#690
bump pyvista[jupyter] from 0.45.1 to 0.45.2	#699
bump tox from 4.25.0 to 4.26.0	#700
bump plotly from 6.0.1 to 6.1.1	#701
bump notebook from 7.4.2 to 7.4.3	#705
bump plotly from 6.1.1 to 6.1.2	#709

Maintenance

update CHANGELOG for v1.4.5	#706

4.21 1.4.5 (May 27, 2025)

Added

add deprecated warning for *index_pattern* search option	#704

4.22 1.4.4 (May 08, 2025)

Added

add metatag content	#680

Fixed

block snowballstemmer <3 for now	#686

Documentation

Update `CONTRIBUTORS.md` with the latest contributors	#669, #677

Dependencies

bump tox from 4.24.2 to 4.25.0	#668
bump pyvista[jupyter] from 0.44.2 to 0.45.0	#675
bump notebook from 7.3.3 to 7.4.1	#676
bump jupytext from 1.16.7 to 1.17.1	#678

Maintenance

add github token	#661
update CHANGELOG for v1.4.0	#662
bump version 1.5.dev0	#663
update CHANGELOG for v1.4.1	#665
bump ansys/actions from 8 to 9 in the actions group	#670

4.23 1.4.3 (May 07, 2025)

Added

add metatag content	#680

Maintenance

bump ansys/actions from 8 to 9 in the actions group	#670

4.24 1.4.2 (March 27, 2025)

Fixed

autoapi child class template	#666

4.25 1.4.1 (March 27, 2025)

Fixed

quarto std logging	#664

4.26 1.4.0 (March 27, 2025)

Added

enable matplotlib and plotly in examples	#567
feat: add python 3.13 support	#635
feat: Add Tox environment for documentation preview	#642
migrate css to scss	#644

Fixed

update the style for highlight name function	#652
update the changelog action only for PR	#654
change logging info to error in quarto build	#655
quarto error logging	#658

Documentation

chore: update CHANGELOG for v1.3.0	#614
chore: update CHANGELOG for v1.3.1	#618
chore: update CHANGELOG for v1.3.2	#631
docs: add contribute section	#636
docs: add contribute page in toctree	#638
chore: update CHANGELOG for v1.3.3	#648

Dependencies

chore(deps): bump jupytext from 1.16.6 to 1.16.7	#622
chore(deps): bump sphinx-autoapi from 3.4.0 to 3.5.0	#623
chore(deps): bump sphinx-gallery from 0.18.0 to 0.19.0	#625
chore(deps): bump pygithub from 2.5.0 to 2.6.0	#626
chore(deps): bump sphinx from 8.1.3 to 8.2.0	#628
chore(deps): bump sphinx-autoapi from 3.5.0 to 3.6.0	#629
build: bump nbsphinx from 0.9.6 to 0.9.7	#637
build: bump tox from 4.24.1 to 4.24.2	#645
build: bump notebook from 7.3.2 to 7.3.3	#651

Maintenance

feat: migrate the builds system to stb	#639

Miscellaneous

chore: bump version 1.4.dev0	#615
fix: remove flake8 configuration file	#641
fix: improve Logging in Quarto cheatsheet build process	#646
cheatsheet and whatsnew functions into separate modules and implement sidebar ordering	#656
ansys sphinx theme variables	#657

4.27 1.3.3 (March 11, 2025)

4.27.1 Dependencies

- chore(deps): bump pygithub from 2.6.0 to 2.6.1 #632

4.27.2 Miscellaneous

- fix: alignment and styles for primary sidebar #621
- fix: typo in autoapi template #630

4.27.3 Documentation

- Fix: whatsnew config instructions #619

4.27.4 Maintenance

- fix: prettier pre-commit hook #627

4.28 1.3.1 (2025-02-06)

4.28.1 Documentation

- Fix: optimize imports for whatsnew #617

4.29 1.3.0 (2025-02-05)

4.29.1 Dependencies

- chore(deps): bump pygithub from 2.4.0 to 2.5.0 #582
- chore(deps): bump pyvista[jupyter] from 0.44.1 to 0.44.2 #589
- chore(deps): bump sphinx-autoapi from 3.3.3 to 3.4.0 #592
- chore(deps): bump notebook from 7.2.2 to 7.3.1 #596
- chore(deps): bump jupytext from 1.16.4 to 1.16.5 #598
- chore(deps): bump jupytext from 1.16.5 to 1.16.6 #600
- chore(deps): bump notebook from 7.3.1 to 7.3.2 #606
- chore(deps): bump nbsphinx from 0.9.5 to 0.9.6 #607
- chore(deps): bump sphinx-notfound-page from 1.0.4 to 1.1.0 #611

4.29.2 Miscellaneous

- chore: bump version 1.3.dev0 #577
- fix: CONTRIBUTORS.md #578
- fix: errors and warnings caused by CSS imports #588
- fix: typo in date variable for last modified #602
- chore: update CHANGELOG for v1.2.5 #609

4.29.3 Documentation

- chore: update CHANGELOG for v1.2.0 #576
- feat: add whatsnew options #583
- chore: update CHANGELOG for v1.2.1 #585
- chore: update CHANGELOG for v1.2.2 #587
- chore: update CHANGELOG for v1.2.3 #595
- chore: update CHANGELOG for v1.2.4 #604

- chore: update CHANGELOG for v1.2.7 #613

4.29.4 Maintenance

- fix: style and license headers #608

4.30 1.2.7 - 2025-02-04

4.30.1 Maintenance

- fix: astroid version #612

4.31 1.2.5 - 2025-01-13

4.31.1 Dependencies

- chore(deps): update pydata-sphinx-theme requirement from <0.16,>=0.15.4 to >=0.15.4,<0.17 #605

4.32 1.2.4 - 2024-12-20

4.32.1 Miscellaneous

- fix: impose upper version for the pydata-sphinx-theme #603

4.33 1.2.3 - 2024-12-05

4.33.1 Miscellaneous

- fix: rename the `date` to `ast_build_date` in js #594

4.34 1.2.2 - 2024-11-21

4.34.1 Miscellaneous

- fix: section ids with search #586

4.35 1.2.1 - 2024-11-13

4.35.1 Miscellaneous

- fix: enable local page *Search* and default to PyData search on 'Enter' #584

4.35.2 Documentation

- Enable 'show_prev_next' in the documented defaults #580

4.36 1.2.0 - 2024-10-31

4.36.1 Fixed

- fix: contributors file #529
- fix: padding for toctree entry in sidebar #554
- fix: revisit header size #555

4.36.2 Dependencies

- chore: bump version 1.2.dev0 #518
- chore(deps): bump sphinx-gallery from 0.17.1 to 0.18.0 #538
- chore(deps): bump sphinx from 8.0.2 to 8.1.3 #546
- chore(deps): update pydata-sphinx-theme requirement from <0.16,>=0.15.4 to >=0.15.4,<0.17 #562
- chore(deps): bump sphinx-autoapi from 3.3.2 to 3.3.3 #569

4.36.3 Miscellaneous

- fix: cleanup autoapi templates using macros #556
- fix: typos #560
- fix: change the display of math rendering #568

4.36.4 Documentation

- chore: update CHANGELOG for v1.1.0 #516
- chore: update CHANGELOG for v1.1.1 #520
- chore: update CHANGELOG for v1.1.2 #522
- docs: adding `title` in example section #524
- fix: add more components to the search indexing #541
- chore: update CHANGELOG for v1.1.3 #545
- chore: update CHANGELOG for v1.1.4 #547
- chore: update CHANGELOG for v1.1.5 #550
- doc: allow developer to skip examples build #553
- chore: update CHANGELOG for v1.1.6 #559
- docs: update the landing page #561
- chore: update CHANGELOG for v1.1.7 #566
- fix: exclude search files #572

4.36.5 Maintenance

- ci: bump ansys/actions from 7 to 8 #530
- chore: add hacktoberfest labels #535
- ci: add new labels #549

- ci: update labeling strategy #558

4.37 1.1.7 - 2024-10-23

4.37.1 Miscellaneous

- fix: ensure app.builder.outdir is a Path object #565

4.38 1.1.6 - 2024-10-18

4.38.1 Fixed

- fix: add the default search options in the config env #557

4.39 1.1.5 - 2024-10-15

4.39.1 Fixed

- use ansys-sphinx-theme variables in CSS files #537

4.40 1.1.4 - 2024-10-14

4.40.1 Fixed

- fix: static search performance #525

4.41 1.1.3 - 2024-10-11

4.41.1 Fixed

- fix: add default *enter* for search #542

4.42 1.1.2 - 2024-10-02

4.42.1 Fixed

- fix: add version as params in cheatsheet #521

4.43 1.1.1 - 2024-10-01

4.43.1 Fixed

- fix(ci): update to Ansys actions v8 #517
- fix: update the token and email of bot for changelog actions #519

4.44 1.1.0 - 2024-10-01

4.44.1 Added

- feat: add static search #487

4.44.2 Changed

- chore: update CHANGELOG for v1.0.0 #445
- chore: update CHANGELOG for v1.0.1 #447
- chore: update CHANGELOG for v1.0.2 #451
- chore: update CHANGELOG for v1.0.3 #455

4.44.3 Fixed

- maint: update tooling, cleanup and drop python 3.9 #484
- feat: implement default search bar expansion behavior #496
- fix: the anchor url of search with the switcher version #497
- fix: change file location for *search.json* file #509
- maint: styles for dark theme search bar #511
- fix: style for smaller media #513
- fix: navigation end columns #514
- fix: add title breadcrumbs #515

4.44.4 Dependencies

- maint: version 1.1.dev0 #448
- build(deps): bump pygithub from 2.3.0 to 2.4.0 #480
- build(deps): bump notebook from 7.2.1 to 7.2.2 #482
- build(deps): bump sphinx-autoapi from 3.2.1 to 3.3.0 #485
- build(deps): bump sphinx-autoapi from 3.3.0 to 3.3.1 #488
- build(deps): bump pandas from 2.2.2 to 2.2.3 #508
- build(deps): bump sphinx-autoapi from 3.3.1 to 3.3.2 #512

4.44.5 Documentation

- chore: update CHANGELOG for v1.0.4 #463
- chore: update CHANGELOG for v1.0.6 #476
- chore: update CHANGELOG for v1.0.7 #478
- chore: update CHANGELOG for v1.0.8 #491
- chore: update CHANGELOG for v1.0.9 #501
- chore: update CHANGELOG for v1.0.10 #504
- chore: update CHANGELOG for v1.0.11 #507

4.45 1.0.11 - 2024-09-19

4.45.1 Fixed

- fix: location of nbsphinx #506

4.46 1.0.10 - 2024-09-18

4.46.1 Fixed

- fix: do not display captions for nbgallery #503

4.47 1.0.9 - 2024-09-16

4.47.1 Added

- feat: add member_order to autoapi #495

4.47.2 Fixed

- fix: `autoapi` relative directory path wrt `tox` env #494

4.48 1.0.8 - 2024-09-03

4.48.1 Fixed

- fix: Align jupyter cell output #489
- fix: the download in sphinx gallery #490

4.49 1.0.7 - 2024-08-23

4.49.1 Fixed

- fix: autoapi extension #472
- fix: admonitions styles for `topic` admonition #477

4.50 1.0.6 - 2024-08-23

4.50.1 Fixed

- fix: download icon with sphinx-gallery and nbsphinx #471
- feat: add different width for different media for main content #473
- fix: the scrollbar on sidebar #474

4.50.2 Documentation

- chore: update CHANGELOG for v1.0.5 #470

4.51 1.0.5 - 2024-08-16

4.51.1 Fixed

- feat: add default logo links for Ansys and PyAnsys logos #469

4.51.2 Dependencies

- build(deps): bump nbsphinx from 0.9.4 to 0.9.5 #465

4.52 1.0.4 - 2024-08-13

4.52.1 Fixed

- fix: tables and cell output #460

4.52.2 Dependencies

- ci: bump ansys/actions from 6 to 7 #457
- build(deps): bump numpydoc from 1.7.0 to 1.8.0 #459

4.53 1.0.3 - 2024-08-09

4.53.1 Fixed

- fix: minor style changes #452
- fix: downgrade the autoapi and keep `autoapi` toctree to `True` by default #453
- fix: *pygment_styles* with dark and light theme and dark theme table #454

4.54 1.0.2 - 2024-08-08

4.54.1 Changed

- maint: update ansys actions #449

4.54.2 Fixed

- fix: sphinx design image background #450

4.55 1.0.1 - 2024-08-08

4.55.1 Fixed

- fix: stable docs indexing package name #446

4.56 1.0.0 - 2024-08-08

4.56.1 Added

- fix: update the github icon #401

- feat: add default logo and update logo option with theme #425
- feat: add quarto cheat sheet extension with cheat sheet option #428

4.56.2 Changed

- chore: update CHANGELOG for v0.16.2 #381
- chore: update CHANGELOG for v0.16.3 #389
- chore: update CHANGELOG for v0.16.4 #390
- chore: update CHANGELOG for v0.16.5 #394
- chore: update CHANGELOG for v0.16.6 #402

4.56.3 Fixed

- fix: Align cheat sheet center #382
- fix: reformat the style files #406
- fix: reformat the table styles #408
- fix: reformat navigation bar and background #409
- fix: *primary* ,`secondary` sidebars and links #411
- fix: sphinx design reformat #412
- fix: update the breadcrumbs #419
- fix: admonitions style #424
- fix: sidebar borders and overflow #427
- fix: search bar styles #429
- fix: updated the logo options #431
- fix: add dropdown styles for the header navigation bar #437
- fix: dark theme variables #438
- fix: sphinx card *box shadow* on focus #439
- fix: focus links with keyboard #440
- fix: search bar style for dark theme, icons links #442

4.56.4 Dependencies

- build(deps-dev): update pydata-sphinx-theme requirement from <0.15,>=0.14 to >=0.15 #336
- chore: version 0.17.dev0 #386
- chore(deps): bump requests from 2.32.2 to 2.32.3 #391
- docs: reformat the documentation #396
- chore(deps): bump sphinx-autoapi from 3.1.1 to 3.1.2 #405
- build(deps): bump pyvista[jupyter] from 0.43.10 to 0.44.0 #413
- build(deps): bump jupytext from 1.16.2 to 1.16.3 #415
- build(deps): bump sphinx from 7.3.7 to 7.4.4 #416

- build(deps): bump sphinx from 7.4.4 to 7.4.5 #417
- build(deps): bump sphinx from 7.4.5 to 7.4.6 #418
- build(deps): bump sphinx-autoapi from 3.1.2 to 3.2.0 #420
- build(deps): bump sphinx-gallery from 0.16.0 to 0.17.0 #421
- build(deps): bump pyvista[jupyter] from 0.44.0 to 0.44.1 #422
- build(deps): bump sphinx from 7.4.6 to 7.4.7 #423
- build(deps): bump sphinx-autoapi from 3.2.0 to 3.2.1 #426
- build(deps): bump sphinx-notfound-page from 1.0.2 to 1.0.3 #432
- build(deps): bump jupytext from 1.16.3 to 1.16.4 #433
- build(deps): bump sphinx-notfound-page from 1.0.3 to 1.0.4 #434
- build(deps): bump sphinx-design from 0.6.0 to 0.6.1 #435
- build(deps): bump sphinx from 7.4.7 to 8.0.2 #436
- build(deps): bump sphinx-gallery from 0.17.0 to 0.17.1 #441

4.56.5 Miscellaneous

- refactor: remove function duplicate #407
- docs: Update *mail id* in README.rst #414

4.57 0.16.6 - 2024-06-18

4.57.1 Fixed

- fix: wrong env var name for PACKAGE_NAME #395

4.58 0.16.5 - 2024-05-31

4.58.1 Fixed

- fix: sphinx design card font size #393

4.59 0.16.4 - 2024-05-29

4.59.1 Added

- feat: adapt package to general PyAnsys repository layout #387

4.59.2 Dependencies

- chore(deps): bump sphinx-design from 0.5.0 to 0.6.0 #383
- chore(deps): bump sphinx-notfound-page from 1.0.1 to 1.0.2 #384
- chore(deps): bump sphinx-autoapi from 3.1.0 to 3.1.1 #385

4.60 0.16.3 - 2024-05-29

4.60.1 Fixed

- fix: update the sphinx design style to disable display of name #388

4.61 0.16.2 - 2024-05-22

4.61.1 Changed

- chore: update CHANGELOG for v0.16.1 #379

4.61.2 Miscellaneous

- docs: update changelog_template.jinja #380

4.62 0.16.1 - 2024-05-22

4.62.1 Added

- feat: add nerd fonts for `autoapi` templates icon #362
- feat: add the changelog action #370
- feat: add autoapi extension #372

4.62.2 Fixed

- fix: add changelog action in ci-cd #378

4.62.3 Dependencies

- chore(deps): bump requests from 2.31.0 to 2.32.1 #374
- maint: update the sphinx-autoapi version #375
- chore(deps): bump sphinx-notfound-page from 1.0.0 to 1.0.1 #376
- chore(deps): bump requests from 2.32.1 to 2.32.2 #377

CHAPTER

FIVE

CONTRIBUTE

Thank you for your interest in contributing to Ansys Sphinx theme! Contributions are welcome to make the project better, whether it's fixing bugs, adding new features, or improving documentation. Below are the guidelines to follow when contributing.

User Learn how to contribute as a user.

Contributing as a user Developer Learn how to contribute as a developer.

Contributing as a developer

5.1 Contributing as a user

Users can contribute in a variety of ways, such as reporting bugs, requesting new features, testing in-development features, starting discussions, answering questions, and sharing their work with the community.

Warning

Do not include any proprietary or sensitive information when reporting bugs or showcasing your work.

Report bugs Found a bug? Report it here.

Report bugs Request a new feature Got an idea for a new feature? Share it!

Request a new feature Test a new feature Anxious to try out a new feature? Here's how you can do it.

Test a new feature Answer questions Help others by answering their questions.

Answer questions Share your work Share your work with the community.

Share your work

5.1.1 Report bugs

If you encounter a bug or an issue while using the project, please report it. Your feedback helps to identify problems.

- Search the Ansys Sphinx theme issues to see if the issue has already been reported.
- Create a new issue if it hasn't been reported.
  - Include a clear description of the problem.
  - Provide steps to reproduce the issue.
  - Mention the version of the project you're using.
  - Include screenshots or logs if possible.

5.1.2 Request a new feature

Do you have an idea for a new feature or an improvement? Your suggestions are welcome. You can request a new feature by creating an issue in the Ansys sphinx theme issues board.

5.1.3 Test a new feature

It is possible to test a new feature before it is officially released. To do so, you can install Ansys Sphinx theme from the source code.

5.1.4 Answer questions

Another great way to contribute is to help others by answering their questions. Maintain a positive and constructive attitude while answering questions. If you don't know the answer, you can still help by pointing the person in the right direction.

5.1.5 Share your work

If you have used Ansys Sphinx theme to create something interesting, share it with the rest of the community. You can share your work in the Ansys Sphinx theme issues. Include a brief description of your work and any relevant links that others may find useful.

5.2 Contributing as a developer

Fork the repository Learn how to fork the project and get your own copy.

Fork the repository Clone the repository Download your own copy in your local machine.

Clone the repository Install for developers Install the project in editable mode.

Install for developers

5.2.1 Fork the repository

Forking the repository is the first step for contributing to the project. This allows you to have your own copy of the project so you can make changes without affection the main project. Once you have made your changes, you can submit a pull-request to the main project to have your changes reviewed and merged.

Fork this project

> **Note**
>
> If you are an Ansys employee, you can skip this step.

5.2.2 Clone the repository

Make sure you configure SSH with your GitHub account. This allows you to clone the repository without having to use tokens or passwords. Also, make sure you have git installed in your machine.

To clone the repository using SSH, run:

```
git clone git@github.com:ansys/ansys-sphinx-theme
```

5.2.3 Install for developers

Installing Ansys sphinx theme in development mode allows you to perform changes to the code and see the changes reflected in your environment without having to reinstall the library every time you make a change.

Virtual environment

Start by navigating to the project's root directory by running:

```
cd ansys-sphinx-theme
```

Then, create a new virtual environment named `.venv` to isolate your system's Python environment by running:

```
python -m venv .venv
```

Finally, activate this environment by running:

Windows

CMD

```
.venv\Scripts\activate.bat
```

PowerShell

```
.venv\Scripts\Activate.ps1
```

macOS/Linux/UNIX

```
source .venv/bin/activate
```

Development mode

Now, install Ansys sphinx theme in editable mode by running:

```
python -m pip install --editable .
```

Verify the installation by checking the version of the library:

```
from ansys_sphinx_theme import __version__
print(f"Ansys sphinx thenme version is {__version__}")
```

```
>>> Ansys sphinx theme version is 1.13.dev0
```

Install `Tox`

Once the project is installed, you can install Tox. This is a cross-platform automation tool. The main advantage of Tox is that it allows you to test your project in different environments and configurations in a temporary and isolated Python virtual environment. To install Tox, run:

```
python -m pip install tox
```

Finally, verify the installation by listing all the different environments (automation rules) for Ansys Sphinx theme:

```
python -m tox list
```

Default Tox environments

Environment	Description	usage
code-style	check project code style	python -m tox -e code-style
doc-style	Checks project documentation style	python -m tox -e doc-style
doc-links	Checks documentation links and pages generates properly	python -m tox -e doc-links
doc-html	Checks documentation links and pages generates properly	python -m tox -e doc-html
doc-pdf	Checks documentation links and pages generates properly	python -m tox -e doc-pdf
doc-clean	Checks documentation links and pages generates properly	python -m tox -e doc-clean
doc-serve	Checks documentation links and pages generates properly	python -m tox -e doc-serve
dist	Checks project distribution	python -m tox -e dist

Adhere to code style

Ansys Sphinx theme follows the PEP8 standard as outlined in PEP 8 in the *PyAnsys Developer's Guide* and implements style checking using pre-commit.

To ensure your code meets minimum code styling standards, run these commands:

```
pip install pre-commit
pre-commit run --all-files
```

or use tox as above:

```
tox -e code-style
```

You can also install this as a pre-commit hook by running this command:

```
pre-commit install
```

This way, it's not possible for you to push code that fails the style checks:

```
$ pre-commit install
$ git commit -am "added my cool feature"
ruff.....................................................................Passed
ruff-format..............................................................Passed
codespell................................................................Passed
prettier.................................................................Passed
check for merge conflicts................................................Passed
debug statements (python)................................................Passed
check yaml...............................................................Passed
trim trailing whitespace.................................................Passed
Validate GitHub Workflows................................................Passed
Add License Headers......................................................Passed
```

Customize the theme

Theme configuration is distributed across multiple files for flexibility and modularity.

Customize the SCSS files

Many styles are defined in SCSS files located in the `src/ansys_sphinx_theme/assets/styles/` directory. The primary SCSS files include:

- `ansys-sphinx-theme.scss`: Contains the main theme configuration.
- `pydata-sphinx-theme-custom.scss`: Contains custom modifications for the PyData theme.

To modify the SCSS files, follow these steps:

1. Navigate to the `src/ansys_sphinx_theme/assets/styles/` directory.
2. Edit the relevant SCSS files as needed.
3. Build the library using the following command along with the documentation dependencies:

   ```
   python -m pip install -e '.[doc]'
   ```

   This command installs the library in editable mode and builds the SCSS files.

   Important

   The built SCSS files are stored in the `src/ansys_sphinx_theme/theme/ansys_sphinx_theme/static/styles/` directory. These files are regenerated during each build process, so avoid editing them directly.

   Only changes made to the `SCSS` source files are considered valid theme updates. Modifying the generated `CSS` files has no impact because the source of truth is the `SCSS` files.

4. Build the documentation and serve it locally using any of the following commands:

   #### using Tox (recommended)

   Linux/macOS/UNIX/Windows

   ```
   python -m tox -e doc-serve
   ```

   #### using Makefile

   #### Linux or macOS

   ```
   make -C doc serve
   ```

   #### Windows

   ```
   doc\make.bat serve
   ```

   #### using stb

   Linux/macOS/UNIX/Windows

   ```
   stb serve doc/source
   ```

   After the build completes, the documentation is served on `localhost` and automatically opens in the default web browser. SCSS file changes are monitored, and the documentation rebuilds automatically.

Note

To use **Tox** for building and serving documentation, run the following command:

```
python -m tox -e doc-serve
```

This command builds the documentation, opens it in the default browser, and monitors the source files for changes to trigger automatic rebuilds.

HTML, CSS, and JavaScript guidelines

When contributing to the theme, follow the same file conventions used throughout the project so changes remain maintainable and easy to review.

HTML templates

- Place custom HTML templates under `src/ansys_sphinx_theme/theme/ansys_sphinx_theme/`.
- Prefer small, focused template overrides instead of large rewrites of the upstream PyData theme.
- Keep template logic readable and maintainable. Avoid embedding complex business logic directly into templates when a Python helper or configuration hook is more appropriate.
- If a change affects page layout or navigation, verify the rendered output in the documentation and check the affected pages visually.

CSS and SCSS

- Keep styles organized under `src/ansys_sphinx_theme/assets/styles/`.
- Prefer editing the source SCSS files instead of the generated CSS bundle in `src/ansys_sphinx_theme/theme/ansys_sphinx_theme/static/styles/`.
- Keep selectors specific and scoped to the theme to reduce unintended side effects.
- When a change introduces a new option, variable, or visual behavior, document it in the relevant user-guide page and update any examples that depend on the behavior.

JavaScript

- Store JavaScript files under `src/ansys_sphinx_theme/theme/ansys_sphinx_theme/static/js/`.
- Keep scripts focused and avoid duplicating logic across files.
- Prefer small, readable functions with clear responsibilities.
- If the behavior affects custom theme interactions, validate the result in the documentation and, when appropriate, add or update tests.

Theme option updates

If a change adds, renames, or modifies a public theme option, configuration key, or supported behavior, make sure to update the appropriate documentation in the user guide and examples. This includes:

- the option documentation in `doc/source/user-guide/options.rst` or the relevant page,
- any examples that rely on the option,
- and any configuration tables or references that describe the theme capabilities.

This requirement applies to new options, new visual defaults, and changes to how users configure the theme.

Build the documentation

To build documentation locally, you can either use Tox as mentioned above or run the following commands:

1. Install the required dependencies by running:

```
python -m pip install -e .[doc]
```

2. Build the documentation by running:

```
# On Linux or macOS
make -C doc/ html

# On Windows
doc\make.bat html
```

3. The documentation is built in the `doc/_build/html` directory. Open the `index.html` file in your browser to view the documentation.

You can clean the build directory by running:

```
# On Linux or macOS
make -C doc/ clean

# On Windows
doc\make.bat clean
```

Note

Use `tox -e doc-serve` to build the documentation and open it in your default browser. This command will also watch for changes in the source files and rebuild the documentation automatically.

PYTHON MODULE INDEX

e

examples, **??**
examples.sample_func, **??**
examples.samples, **??**
examples.type_hint_example, **??**